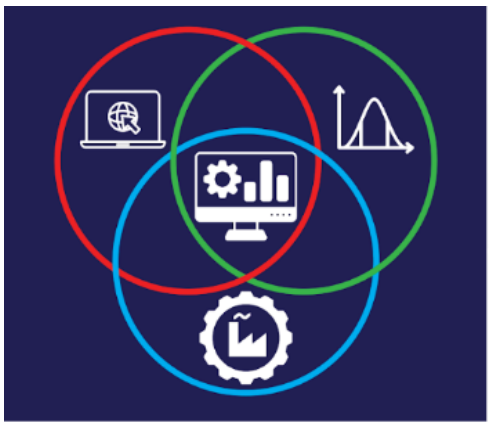
Data Science Textbook
Henrico County Public Schools



Unit 1: Data Literacy
- The Data Cycle
- Types of Data
- Data Questions
- Background Research

Unit 2: Visualizations
- Histograms & Boxplots
- Scatterplots & Line Graphs
- Pie Charts & Bar Charts
- Heatmaps & Other Visualizations
- Choosing Visualization Types
- Data Processing
- Data Cleaning
  - `pandas` cheat sheet
- Descriptive Statistics
- Statistics in Python
- Creating Visualizations
- Advanced Visualizations
  - `matplotlib` cheat sheet
- Documentation

Unit 3: Mathematical Models
- Linear Regression
  - Polynomial Regression
- Logistic Regression
  - Multiple Predictors
- Noise & Overfitting
- Normal Distribution
- Decision Trees
- Algorithm Selection

Resources
- Data Question Flowchart
- Data Question Distiller
- Attribute Exploration Sheet
- VS Code Directions
- Data Sources List

Updated August 2025 | Sara Barbolish

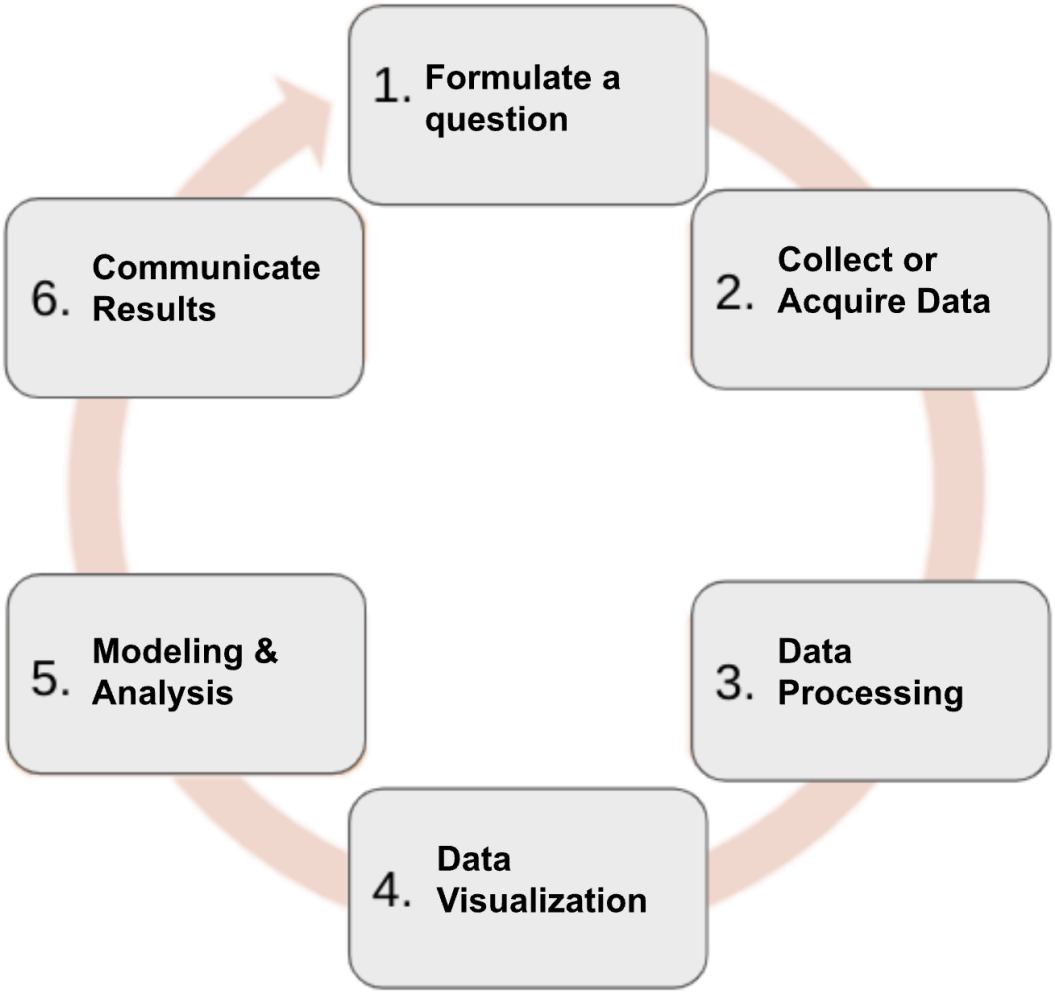
UNIT 1: Data Science for All

Standard 1: Data Questions and the Data Cycle (DS.1)	Not Yet	Almost	Yes
Learning Experience #1: Data Questions			
I can **create** a research question for data analysis			
I can **develop** questions for an exploratory data analysis based on a data set or multiple data sets			
Learning Experience #2: The Data Cycle			
I can **identify** the steps of the data cycle			
I can **use** the steps of the data cycle (except parts of analysis) to explore data			

Standard 2: Storytelling (DS.3, DS.5)	Not Yet	Almost	Yes
Learning Experience #1: Data Literacy			
I can **interpret** the meaning of a data-heavy report			
I can **interpret** and **critique** visualizations created by a variety of sources			
Learning Experience #2: Storytelling			
I can **combine** data resources and visualizations to tell a data story that can **describe** something in the real world.			

The Data Cycle

The Data Cycle	In the data cycle, a data scientist will ask a question, collect or acquire the data needed to answer that question, perform a data analysis (including pre-processing and processing, visualizations, models, and general analysis) and then communicate their findings. Once they have communicated their findings, they will notice that new questions have been brought to light. These may come in the form of a further question, the need for an additional attribute of the data, the need for different data (different people, different place, different time, etc), or a critique of the data analysis process. Once that question is created, the data cycle is repeated.



Types of Data

	Definition	***Example***	***Questions to Ask***	***Notes***
Quantitative / Numeric	Quantitative data uses numbers to describe an amount of something. Measures like mean and median would make sense with this data. Basic arithmetic would also make sense with this type of data	age, year, number of pets, height	What is "normal"? What is the range of the data? How "different" is one data point? These questions ask about the *spread of the data*	Not all data using numbers is quantitative. For example "tv channel" or "ID number" would not be quantitative.
Ordinal	Ordinal data is data that can be put in an order. Quantitative data is a type of ordinal data, but ordinal data does not need to be numeric.	Ordinal data often has to do with 'rating'. For example... • Strongly disagree, disagree, agree, strongly agree • Poor, good, great • On a scale of 1 to 10, how much does the injury hurt? Dates may also be considered ordinal	What is "normal"? What is the range of the data? How "different" is one data point? These questions ask about the *spread of the data*	
Categorical	Categorical data puts respondents into groups.	Categorical data is often collected using a multiple choice or multiple answer question. It cannot be ordered. For example, favorite season breaks respondents into "spring", "summer", "fall", and "winter".	What is most common? What is the makeup of the data? These questions ask about the *composition of the data*	Some 'categories' would require an 'other' in order to categorize. This is particularly true of categories like 'race', where people differ a lot. It is important to consider how many people would fall into the 'other' category. If it would be a large number of respondents, consider collecting qualitative data instead.
Qualitative	Qualitative data is typically words and descriptions. These types of questions are useful when you can't clearly categorize questions.	These are used for open-ended questions. For example: "what was your favorite part of this week"? Or "If you could have any superpower, what would it be?"	How do people feel about this? Are there patterns in the data? These questions ask about *patterns and descriptions in the data*	

Data Questions

Term	Definition
Data Question	A Data Question is a question that can be answered with data and facilitate a quality data analysis. A data question might arise from a *broad question* or a *subjective question*. Answering the question allows further questions to arise. Answering the question should contribute to a larger understanding of the world or an overarching question.
Research Project	A research project follows the traditional data cycle: 1. Choose a topic 2. Ask a specific question 3. Collect data 4. Perform a data analysis 5. Draw a conclusion 6. Ask a new question
Exploratory Data Analysis	The data scientist must "make sense of" the data. This makes a different data cycle: 1. Acquire or Collect Data 2. Explore the Data 3. Ask a specific question 4. Perform a data analysis 5. Draw a conclusion 6. Ask a new question
Specific	A *specific question* is one that is clear and/or well defined. ❌Non-Example: Are girls more successful than boys? ⭐Example: Do girls earn higher high school GPAs than boys?
Objective	An *objective question* is one that is not looking for an individual's/ the data scientist's opinion. To capture opinion: focus on public perception of the question. ❌Non-Example:What is the best book ever written? ⭐Example: What books are commonly people's favorites?
Fruitful	A *fruitful question* is one that cannot be answered in just a few sentences. A question that is *not* fruitful has one simple answer/explanation and can be looked up / It is a fact or figure. ❌Non-Example: How many people live in the United States? ⭐Example: How has the population of the United States fluctuated over time?

Be careful not to treat "fruitful" and "specific" as opposites.

	Not Fruitful	***Fruitful***
Specific	❌ Of all rollercoasters in the United States that are currently operational, which rollercoaster has the fastest maximum speed?	⭐Considering all rollercoasters in the United States that are currently operational, what factors correlate with maximum speed? Are these correlations different within the top 10% fastest roller coasters?
Not Specific	❌ Which rollercoaster is the fastest?	❌ Are there things about the fastest roller coasters that make them different?

Background Research Reliability

RELIABILITY CHECKLIST

Make sure you can check all of these boxes before using a source!

Category	Questions
Currency	☐ When was the site created and last updated? ☐ Do all the links on the site work?
Relevance	☐ Looking past the headline – does the information relate to your purposes? ☐ Will it help you?
Authority	☐ Does the site have an author or an organization that runs it? ☐ Is the author or organization reliable? ☐ Can the public edit the site? (Be careful if it can.) ☐ What is the domain? (You can trust ".gov" and ".edu." Use caution with ".org" and ".com.")
Accuracy	☐ Does the site have a disclaimer? (If so, stories may not be true.) ☐ Is the site biased because it is promoting a cause? ● The domain ".org" promotes non-profit causes ☐ Are there any typos or grammatical errors? (You don't want to see any.)
Purpose	☐ Is the content factual or opinionated? ☐ Are the opinions supported with facts? ☐ Does the site list its resources for the information presented?

GENERAL RULES

Reliable and Helpful Sites	What to Avoid
● Snopes - the best source for factual information about rumors ● Hoax-Slayer - email hoaxes & Internet scams ● Urban Legends and Folklore from About.com ● Truth or Fiction ● Straight Dope - mission is to fight ignorance ● Issues and Controversies (Clever) ● Britannica.com/procon ● **Reliable News**: NPR, CNN, *Washington Post, London Times, Oregonian,* BCC, and Reuters ● For more information and a list of tested sites click here.	● Websites that end in "lo" ex: Newslo. ● Common news websites that end in ".com.co" ● News stories that known/reputable news sites are not also reporting on. ● Odd domain names ● Bloggers– ex: BuzzFeed Community Posts, Kinja blogs, *Forbes* blogs ● Bad web design and use of ALL CAPS

UNIT 2: Visualizations

Standard 1: Visualizations (DS.6)	**Not Yet**	**Almost**	**Yes**
Learning Experience #1: Visualization Effectiveness			
I can **evaluate** the effectiveness of a visualization and **identify** what aspects of a visualization contribute to the effectiveness			
Learning Experience #2: Emerging Visualizations			
I can **interpret** emerging visualizations, which show any number of attributes			
I can **describe** the advantages and disadvantages of a given emerging visualization			
Learning Experience #3: Choosing Visualizations			
I can **choose** and **create** the most effective visualization given my data. For example, 1. I can **differentiate** data best represented as a scatter plot versus a line graph 2. I can **differentiate** data best represented as a histogram versus a bar chart.			

Standard 2: Data Wrangling (DS.8, DS.12)	**Not Yet**	**Almost**	**Yes**
Learning Experience #1: Choosing Data			
I can **find** high quality data to use for data science projects			
Learning Experience #2: Importing Data			
I can **import** data in a csv document into a Python notebook for analysis			
I can **create** a data frame using the data from my file			
I can **subdivide** datasets in order to analyze with additional attributes			

Standard 3: Data Cleaning (DS.8, DS.12)	**Not Yet**	**Almost**	**Yes**
Learning Experience #1: Data Cleaning Theories			
I can **describe** the common problems in “messy data” and their solutions			
I can **identify** data cleaning decisions that introduce bias & avoid it in my analysis			
Learning Experience #2: Data Cleaning Practice			
I can clean data by **omitting** missing values, without introducing bias			
I can **clean** data in other ways as needed.			

Standard 4: Scatterplots and Line Graphs (DS.10, DS.13)	**Not Yet**	**Almost**	**Yes**
Learning Experience #1: Scatterplot and Line Graph Interpretation			
I can **interpret** a scatterplot and, if appropriate, use it to make predictions			
I can **interpret** scatterplots and line graphs that show additional attributes and use those attributes to **compare**.			
Learning Experience #1: Scatterplot and Line Graph Creation			
I can **create** well labeled and organized scatter plots and line graphs in Python			
I can **subdivide** a dataframe to **create** a well labeled and organized plot			

Standard 5: Histograms and Box Plots (DS.10, DS.13)	**Not Yet**	**Almost**	**Yes**
Learning Experience #1: Histogram and Box Plot Interpretation			
I can use a histogram to **estimate** the mean and its relationship to the median.			
I can **interpret** a histogram			
I can use box plots to **identify** measures of central tendency			
Learning Experience #2: Histogram and Box Plot Creation			
I can **create** a well labeled and organized histogram in Python			

Standard 6: Pie Charts and Bar Graphs (DS.10, DS.13)	**Not Yet**	**Almost**	**Yes**
Learning Experience #1: Pie Chart and Bar Chart Interpretation			
I can **interpret** pie charts and other representations of parts of a whole			
I can **interpret** proportional representations including heat maps and bubble charts.			
Learning Experience #2: Pie Chart and Bar Chart Creation			
I can **create** a well labeled and organized pie or donut chart in Python			
I can **create** a well labeled and organized bar chart in Python			

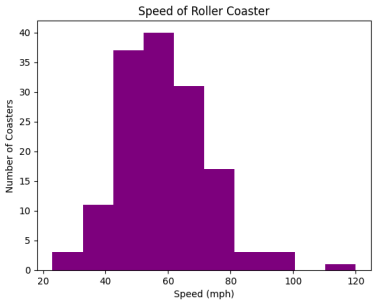
Histograms and Boxplots

Histogram

Number & Type of Attributes: One Numeric Attribute

Purpose: Shows the spread of data

Possible Data Question: How tall is a typical 12-year old?

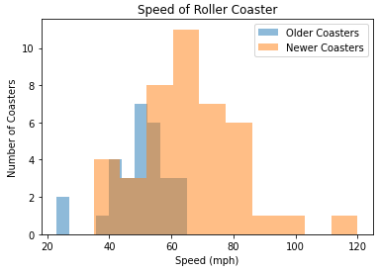


Stacked Histogram

Number & Type of Attributes: One Numeric Attribute, One Categorical Attribute

Purpose: Shows the spread of data, Compares two subsets

Possible Data Question: How tall is a typical 12-year old? Is that different for girls and boys?

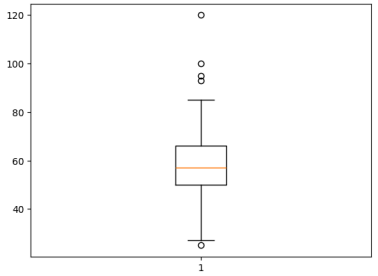


Box and Whisker

Number & Type of Attributes: One Numeric Attribute

Purpose: Shows the spread of data

Possible Data Question: How tall is a typical 12-year old?

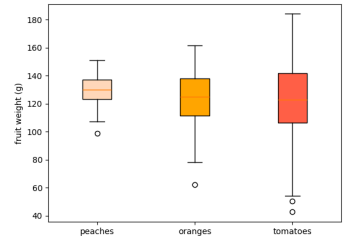


Side-by-Side Boxplot

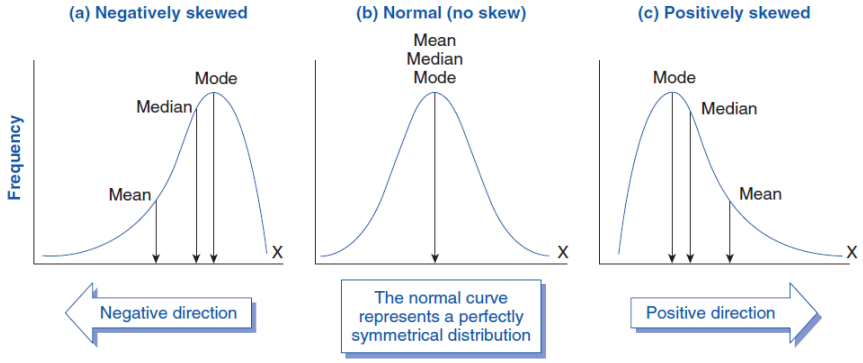
Number & Type of Attributes: One Numeric Attribute, One Categorical Attribute

Purpose: Shows the spread of data, Compares two subsets

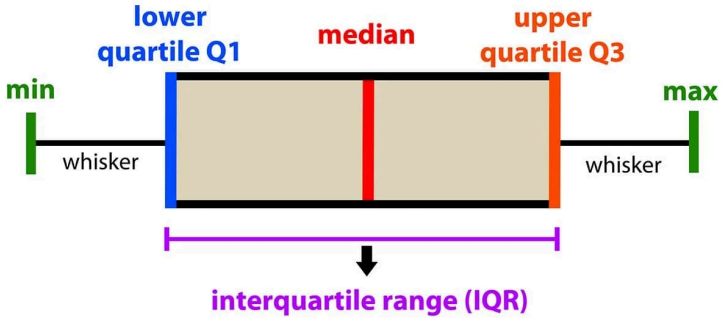
Possible Data Question: How tall is a typical 12-year old? Is that different for girls and boys?



Note: Histograms



Note: Box Plots



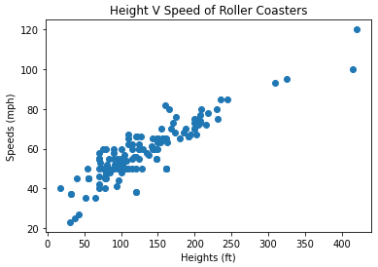
Scatterplots and Line Graphs

Scatterplot

Number & Type of Attributes: Two Numeric Attributes

Purpose: Shows the relationship between two variables

Possible Data Question: What is the relationship between a person's height and their weight?

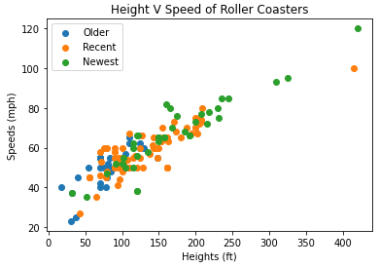


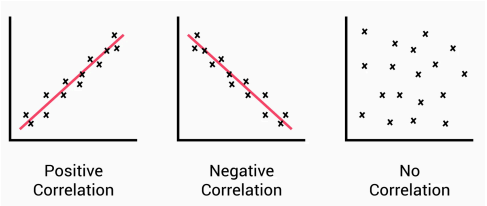
Subsetted Scatterplot

Number & Type of Attributes: Two Numeric Attributes, One Categorical Attribute

Purpose: Shows the relationship between three variables

Possible Data Question: What is the relationship between a person's height and weight? Does that vary for girls and boys?





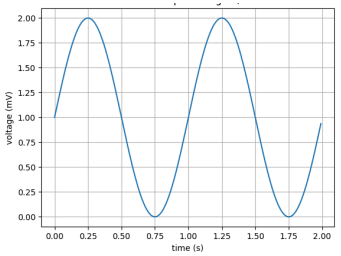
Note: Additional attributes can be added using the size (numeric) or shape (categorical) of each dot.
Using a scale, the colors could also represent a numeric variable

Line Graph

Number & Type of Attributes: Two numeric Attributes (one is *continuous*)

Purpose: Shows the relationship between two variables. Really good for time.

Possible Data Question: How have gas prices varied over time?

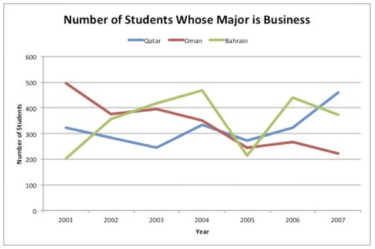


Overlaid Line Graph

Number & Type of Attributes: Two Numeric Attributes (one is *continuous*), One Categorical Attribute

Purpose: Shows the relationship between three variables

Possible Data Question: How have gas prices varied over time in VA, WV, and NC?

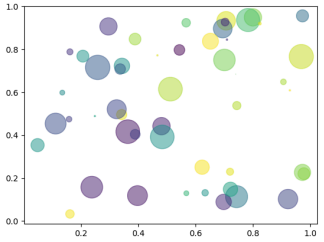


Scatter Bubble Chart

Number & Type of Attributes: Three Numeric Attributes, Optional One Categorical Attribute

Purpose: Shows the relationship between (up to) four variables

Possible Data Question: What is the relationship between a person's height and their weight? Does that vary with age?

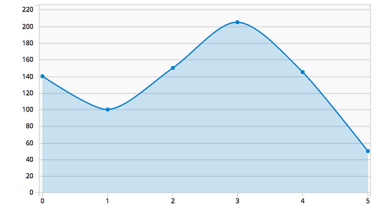


Area Chart

Number & Type of Attributes: Two numeric Attributes (one is *continuous*)

Purpose: Shows the relationship between two variables. Really good for time.

Possible Data Question: How have gas prices varied over time?



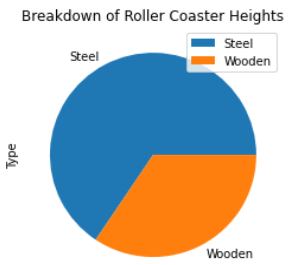
Pie Charts and Bar Charts

Pie Chart

Number & Type of Attributes: One Categorical Attribute

Purpose: Shows the composition of data

Possible Data Question: What portion of people prefer summer, and what portion prefer winter?

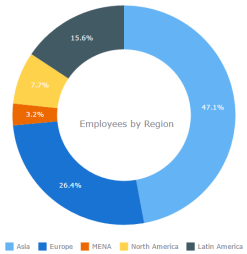


Donut Chart

Number & Type of Attributes: One Categorical Attribute

Purpose: Shows the composition of data

Possible Data Question: What portion of people prefer summer, and what portion prefer winter?

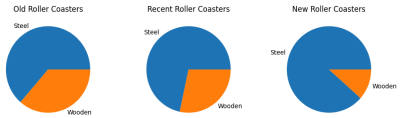


Side-by-Side Pie Charts

Number & Type of Attributes: Two Categorical Attributes

Purpose: Shows the composition of data

Possible Data Question: What portion of people prefer summer, and what portion prefer winter, broken down by age?

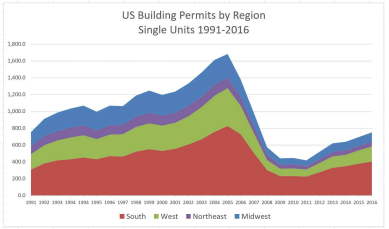


Stacked Area Chart

Number & Type of Attributes: One Numeric (continuous) and one categorical Attribute

Purpose: Shows the composition of data and change over time

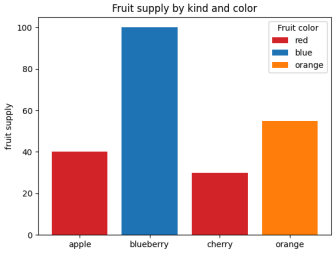
Possible Data Question: How has the number of cars sold change over time, by brand?



Bar Chart (Could be vertical or horizontal)

Number & Type of Attributes: One unique Categorical Attribute & Numeric or just One Categorical

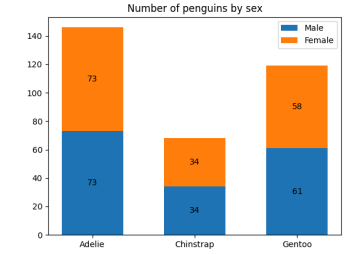
Purpose: Compares the amount of something
Possible Data Question: What is the most popular TV show?



Stacked Bar Chart

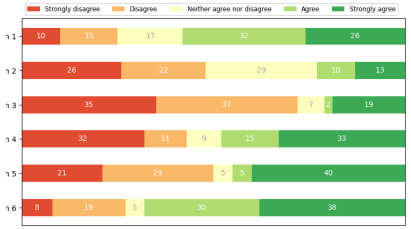
Number & Type: One unique Categorical, One Non-unique categorical, One Numeric

Purpose: Compares the amount of something and breaks down by composition
Possible Data Question: What is the most popular TV show? Are there changes in TV show preference based on your gender?



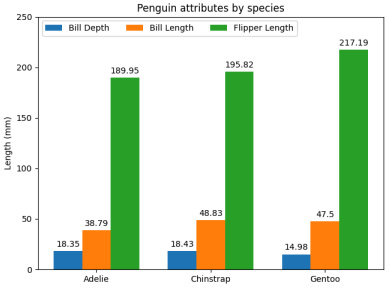
Percentage Bar Chart

Number & Type of Attributes: Two Categorical Variables
Purpose: Shows and compares the composition of data
Possible Data Question: What portion of people prefer summer, and what portion prefer winter? How does that compare by grade?



Grouped Bar Chart

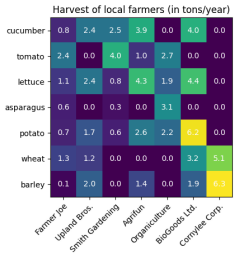
Number & Type of Attributes: One unique Categorical, One Non-unique categorical, One Numeric
Purpose: Compares the amount of something with subsets
Possible Data Question: What is the most popular TV show? Are there changes in TV show preference based on your age group?



Heatmaps and Other Visualizations

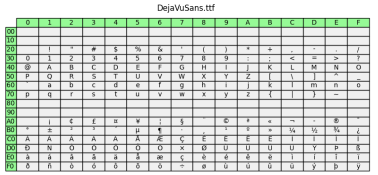
Heatmap

Number & Type of Attributes: Two Categorical Attributes, One Numeric Attribute
Purpose: Shows the relationship between three variables
Possible Data Question: How does the brand of a car and its color affect the price?



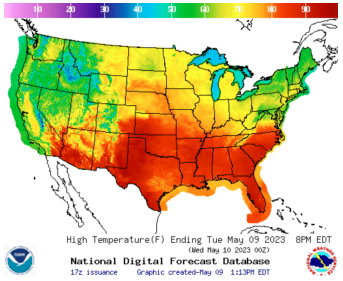
Table

Number & Type of Attributes: Any Number or Type of Attribute
Purpose: Shows all data or a subset of the data in detail
Possible Data Question: What is the relationship between a person's height and their weight? Does that vary for girls and boys?



Geographic Heatmap

Number & Type of Attributes: One Categorical, Location (usually latitude & longitude)
Purpose: Compares a variable by location
Possible Data Question: What parts of the country have the highest unemployment rates?



Matrix

Number & Type of Attributes: Any Number categorical attributes
Purpose: Shows counts of all data or a subset of the data in detail
Possible Data Question: How many people are unemployed? How does that break down by race, gender, and age?

Trade-Off Analysis—Effect Matrix

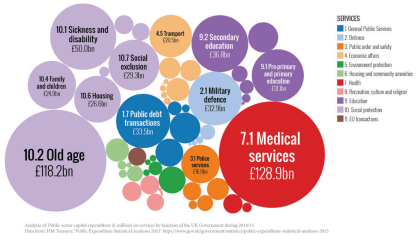
	Safeguard Option 1	*Safeguard Option 2*	*Safeguard Option 3*	*Safeguard Option 4*
Security Criterion A	+7	+6	+5	+8
	8	8	8	8
Security Criterion B	+2	0	0	+3
	9	9	9	9
Security Criterion C	–3	0	–2	+2
	7	7	7	7

Proportion / Bubble Chart

Number & Type of Attributes: One Categorical, Location (usually latitude & longitude)

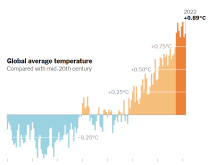
Purpose: Compares the amount of something

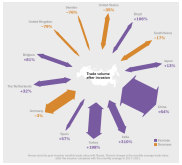
Possible Data Question: What is the most popular TV show?

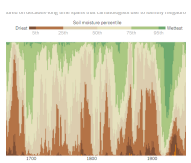


Other Visualizations

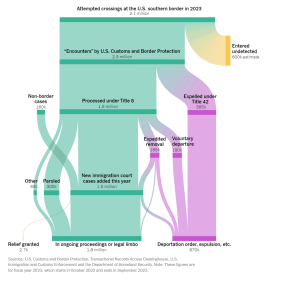
Most visualizations you will see in real data analyses are a combination or adaptation of these types of visualizations. When coding, check the documentation or graph galleries like https://python-graph-gallery.com/. Here are some examples (NYT).

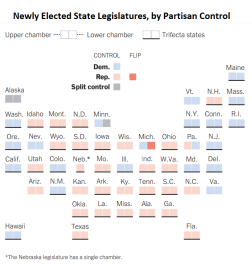


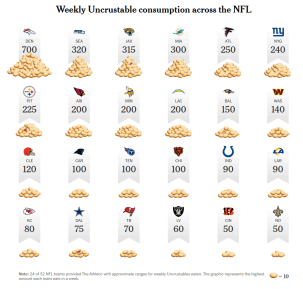




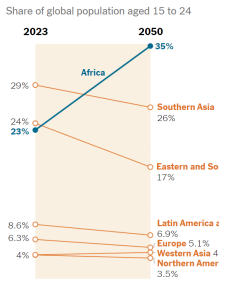


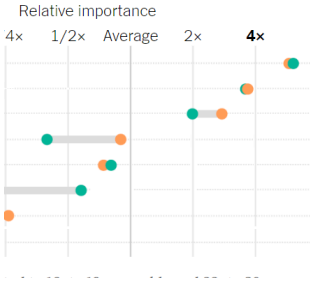














What Graph Can I Use?

Visualization	*Attributes*
One Attribute	
Histogram	One Numeric Attribute
Box and Whisker	One Numeric Attribute
Pie Chart	One Categorical Attribute
Donut Chart	One Categorical Attribute
Bar Chart	One Categorical Attribute
Two Attributes	
Stacked Histogram	One Numeric Attribute, One Categorical Attribute
Side-by-Side Box Plots	One Numeric Attribute, One Categorical Attribute
Scatterplot	Two Numeric Attributes
Line Graph	Two Numeric Attributes (one is continuous)
Area Chart	Two Numeric Attributes (one is continuous)
Side-by-Side Pie Charts	Two Categorical Attributes
Stacked or Grouped Bar Charts	Two Categorical Attributes
Percentage Bar Chart	Two Categorical Variables
Proportion / Bubble Chart	One Numeric, One Categorical (optional second categorical)
Three or More Attributes	
Stacked or Overlaid Area Chart	Two Numeric (one continuous) and one categorical Attribute
Subsetted Scatterplot	Two Numeric Attributes, One Categorical Attribute
Overlaid Line Graph	Two Numeric Attributes (one is continuous), One Categorical
Scatter Bubble Chart	Three Numeric Attributes, Optional One Categorical Attribute
Heatmap	Two Categorical Attributes, One Numeric Attribute
Geographic Heatmap	One Numeric and Location (usually latitude & longitude)
Table	Any Number & Any Type
Matrix	Any Number of Categorical Attributes

This list does <u>not</u> include all possible visualizations!

Data Processing

PYTHON STEPS

Import Libraries	`import pandas as pd`
Read in the data	`my_df = pd.read_csv('./NAME OF FILE.csv')`

Explore the Data	`my_df.head()`
	`my_df.info()`
	`my_df.describe()`

Isolate one attribute	`one_attribute = my_df['Attribute Name']` `print(one_attribute)`

KEY VOCABULARY

`.head()`	Shows the first 5 rows of a dataframe
`.info()`	Shows each attribute, the number of cases that have a value for that attribute, and the data type of the attribute: • `int64`: a number with no decimal • `float64`: a number with a decimal • `object`: a word
`.describe()`	Shows the descriptive statistics (mean, median, mode, maximum, minimum, quartiles, standard deviation) of all attributes in the dataset.
Attribute	An "attribute" is a column of a table. It is a piece of information that describes each case. Most of the time, each case will have multiple attributes

EXAMPLE

```
import pandas as pd
roller_coaster_df = pd.read_csv('./roller_coaster.csv')
print(roller_coaster_df.head())
print(roller_coaster_df.info())
print(roller_coaster_df.describe())
heights = roller_coaster_df['Max_Height']
print(heights)
```

Practice Assignment

Data Cleaning

PYTHON COMMANDS

`.to_numeric()`	Changes all values to numbers	`.str.strip`	Gets rid of white space

`.loc()`	Find and change specific values	`.str.upper`
		`.str.lower`
		`.str.title`
			Makes words UPPER CASE, lower case, or Title Case

`.to_datetime()`	Puts all dates into the same format	`.map()`	Replaces values into categories
`.replace()`	Replaces specific values	`np.nan`	Standard “missing”
`coerce = True`	Add this when you are changing data types	`.dropna()`	Drops lines with any missing values
`.drop_duplicates()`		If more than one line is exactly the same, drops copies	

KEY VOCABULARY

Missing at Random / Missing not at Random	Data is missing not at random if there is some reason, other than chance, that a field was left blank. For example, asking “are you a citizen?” may result in values that are missing not at random and should not be dropped.

EXAMPLE (code on next page)

name	Grade	email	submission_date	passed
Alice Smith	88	alice_smith@gmailcom	2023-05-01	Yes
bob jones	92	bob.jones@gmail.com	2023/05/03	y
Charlie Zhang		charliez@gmail.com	05-04-2023	TRUE
Dana O'Brien	74	dana.obrien@@gmail.com	May 5 2023	1
Eli	105	eli123@gmail.com	NULL	No
John Smith	twenty	jws12@gmail.com	2023/05/03	
Alice Smith	88	alice_smith@gmailcom	2023-05-01	Yes

```python
import pandas as pd
import numpy as np

df = pd.read_csv("students_grades.csv")

# Strip whitespace and standardize capitalization for names
df['name'] = df['name'].str.strip().str.title()

# Convert grades to numeric
df['Grade'] = pd.to_numeric(df['Grade'], errors='coerce')

# Cap grades between 0 and 100
df.loc[df['Grade'] > 100, 'Grade'] = 100
df.loc[df['Grade'] < 0, 'Grade'] = 0

# Fix common email typos using replace
df['email'] = df['email'].str.replace('@@', '@')
df['email'] = df['email'].str.replace('gmailcom', 'gmail.com')

# Convert submission_date to datetime
df['submission_date'] = pd.to_datetime(df['submission_date'], errors='coerce')

# Normalize 'passed' column
passed_map = {
   'yes': True, 'y': True, '1': True, 'true': True,
   'no': False, 'n': False, '0': False, 'false': False
}
df['passed'] = df['passed'].str.lower().map(passed_map)

# Replace missing "passed" rows (np.nan) with False
df['passed'] = df['passed'].replace({np.nan: False})

# Remove duplicate rows
df = df.drop_duplicates()

# Drop rows with missing critical values (name, email, grade)
df = df.dropna(subset=['name', 'email', 'Grade'])
```

Practice Assignment 🔑

Data Wrangling

with pandas Cheat Sheet

http://pandas.pydata.org

Pandas API Reference Pandas User Guide

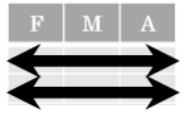
Tidy Data – A foundation for wrangling in pandas

In a tidy data set:



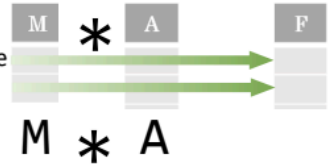
Each **variable** is saved in its own **column**

&



Each **observation** is saved in its own **row**

Tidy data complements pandas's **vectorized operations**. pandas will automatically preserve observations as you manipulate variables. No other format works as intuitively with pandas.



Creating DataFrames

	a	b	c
1	4	7	10
2	5	8	11
3	6	9	12

```
df = pd.DataFrame(
        {"a" : [4, 5, 6],
         "b" : [7, 8, 9],
         "c" : [10, 11, 12]},
       index = [1, 2, 3])
```

Specify values for each column.

```
df = pd.DataFrame(
    [[4, 7, 10],
     [5, 8, 11],
     [6, 9, 12]],
    index=[1, 2, 3],
    columns=['a', 'b', 'c'])
```

Specify values for each row.

		a	b	c
N	v			
D	1	4	7	10
	2	5	8	11
e	2	6	9	12

```
df = pd.DataFrame(
        {"a" : [4 ,5, 6],
         "b" : [7, 8, 9],
         "c" : [10, 11, 12]},
index = pd.MultiIndex.from_tuples(
        [('d', 1), ('d', 2),
         ('e', 2)], names=['n', 'v']))
```

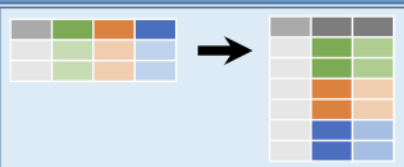
Create DataFrame with a MultiIndex

Method Chaining

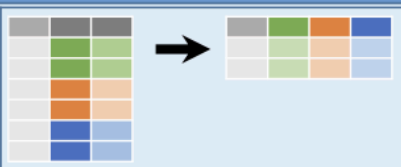
Most pandas methods return a DataFrame so that another pandas method can be applied to the result. This improves readability of code.

```
df = (pd.melt(df)
        .rename(columns={
                'variable':'var',
                'value':'val'})
        .query('val >= 200')
    )
```

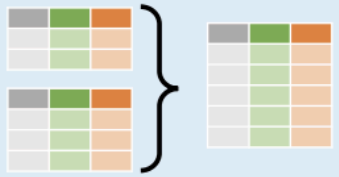
Reshaping Data – Change layout, sorting, reindexing, renaming



pd.melt(df)
Gather columns into rows.



df.pivot(columns='var', values='val')
Spread rows into columns.



pd.concat([df1,df2])
Append rows of DataFrames



pd.concat([df1,df2], axis=1)
Append columns of DataFrames

df.sort_values('mpg')
Order rows by values of a column (low to high).

df.sort_values('mpg', ascending=False)
Order rows by values of a column (high to low).

df.rename(columns = {'y':'year'})
Rename the columns of a DataFrame

df.sort_index()
Sort the index of a DataFrame

df.reset_index()
Reset index of DataFrame to row numbers, moving index to columns.

df.drop(columns=['Length', 'Height'])
Drop columns from DataFrame

Subset Observations - rows



df[df.Length > 7]
Extract rows that meet logical criteria.

df.drop_duplicates()
Remove duplicate rows (only considers columns).

df.sample(frac=0.5)
Randomly select fraction of rows.

df.sample(n=10) Randomly select n rows.

df.nlargest(n, 'value')
Select and order top n entries.

df.nsmallest(n, 'value')
Select and order bottom n entries.

df.head(n)
Select first n rows.

df.tail(n)
Select last n rows.

Subset Variables - columns



df[['width', 'length', 'species']]
Select multiple columns with specific names.

df['width'] *or* **df.width**
Select single column with specific name.

df.filter(regex='*regex*')
Select columns whose name matches regular expression *regex*.

Using query

query() allows Boolean expressions for filtering rows.

```
df.query('Length > 7')
df.query('Length > 7 and Width < 8')
df.query('Name.str.startswith("abc")',
         engine="python")
```

Subsets - rows and columns

Use **df.loc[]** and **df.iloc[]** to select only rows, only columns or both.
Use **df.at[]** and **df.iat[]** to access a single value by row and column.
First index selects rows, second index columns.

df.iloc[10:20]
Select rows 10-20.

df.iloc[:, [1, 2, 5]]
Select columns in positions 1, 2 and 5 (first column is 0).

df.loc[:, 'x2':'x4']
Select all columns between x2 and x4 (inclusive).

df.loc[df['a'] > 10, ['a', 'c']]
Select rows meeting logical condition, and only the specific columns .

df.iat[1, 2] Access single value by index

df.at[4, 'A'] Access single value by label

Logic in Python (and pandas)

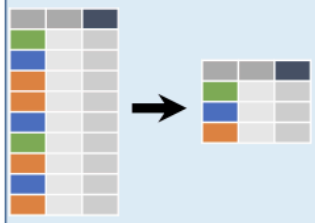
<	Less than	!=	Not equal to
>	Greater than	df.column.isin(*values*)	Group membership
==	Equals	pd.isnull(*obj*)	Is NaN
<=	Less than or equals	pd.notnull(*obj*)	Is not NaN
>=	Greater than or equals	&,|,~,^,df.any(),df.all()	Logical and, or, not, xor, any, all

regex (Regular Expressions) Examples

'\.'	Matches strings containing a period '.'
'Length$'	Matches strings ending with word 'Length'
'^Sepal'	Matches strings beginning with the word 'Sepal'
'^x[1-5]$'	Matches strings beginning with 'x' and ending with 1,2,3,4,5
'^(?!Species$).*'	Matches strings except the string 'Species'

Cheatsheet for pandas (http://pandas.pydata.org/ originally written by Irv Lustig, Princeton Consultants, inspired by Rstudio Data Wrangling Cheatsheet

Group Data



`df.groupby(by="col")`
Return a GroupBy object, grouped by values in column named "col".

`df.groupby(level="ind")`
Return a GroupBy object, grouped by values in index level named "ind".

All of the summary functions listed above can be applied to a group.
Additional GroupBy functions:

`size()`
Size of each group.

`agg(function)`
Aggregate group using function.

The examples below can also be applied to groups. In this case, the function is applied on a per-group basis, and the returned vectors are of the length of the original DataFrame.

`shift(1)`
Copy with values shifted by 1.

`rank(method='dense')`
Ranks with no gaps.

`rank(method='min')`
Ranks. Ties get min rank.

`rank(pct=True)`
Ranks rescaled to interval [0, 1].

`rank(method='first')`
Ranks. Ties go to first value.

`shift(-1)`
Copy with values lagged by 1.

`cumsum()`
Cumulative sum.

`cummax()`
Cumulative max.

`cummin()`
Cumulative min.

`cumprod()`
Cumulative product.

Summarize Data

`df['w'].value_counts()`
Count number of rows with each unique value of variable

`len(df)`
of rows in DataFrame.

`df.shape`
Tuple of # of rows, # of columns in DataFrame.

`df['w'].nunique()`
of distinct values in a column.

`df.describe()`
Basic descriptive and statistics for each column (or GroupBy).

`df.info()`
Prints a concise summary of the DataFrame.

`df.memory_usage()`
Prints the memory usage of each column in the DataFrame.

`df.dtypes()`
Prints a Series with the dtype of each column in the DataFrame.



pandas provides a large set of summary functions that operate on different kinds of pandas objects (DataFrame columns, Series, GroupBy, Expanding and Rolling (see below)) and produce single values for each of the groups. When applied to a DataFrame, the result is returned as a pandas Series for each column. Examples:

`sum()`
Sum values of each object.

`count()`
Count non-NA/null values of each object.

`median()`
Median value of each object.

`quantile([0.25,0.75])`
Quantiles of each object.

`apply(function)`
Apply function to each object.

`min()`
Minimum value in each object.

`max()`
Maximum value in each object.

`mean()`
Mean value of each object.

`var()`
Variance of each object.

`std()`
Standard deviation of each object.

Handling Missing Data

`df.dropna()`
Drop rows with any column having NA/null data.

`df.fillna(value)`
Replace all NA/null data with value.

Make New Columns



`df.assign(Area=lambda df: df.Length*df.Height)`
Compute and append one or more new columns.

`df['Volume'] = df.Length*df.Height*df.Depth`
Add single column.

`pd.qcut(df.col, n, labels=False)`
Bin column into n buckets.



pandas provides a large set of **vector functions** that operate on all columns of a DataFrame or a single selected column (a pandas Series). These functions produce vectors of values for each of the columns, or a single Series for the individual Series. Examples:

`max(axis=1)`
Element-wise max.

`min(axis=1)`
Element-wise min.

`clip(lower=-10,upper=10)`
Trim values at input thresholds

`abs()`
Absolute value.

Windows

`df.expanding()`
Return an Expanding object allowing summary functions to be applied cumulatively.

`df.rolling(n)`
Return a Rolling object allowing summary functions to be applied to windows of length n.

Combine Data Sets

adf

x1	x2
A	1
B	2
C	3

+

bdf

x1	x3
A	T
B	F
D	T

=

Standard Joins

x1	x2	x3
A	1	T
B	2	F
C	3	NaN

`pd.merge(adf, bdf, how='left', on='x1')`
Join matching rows from bdf to adf.

x1	x2	x3
A	1.0	T
B	2.0	F
D	NaN	T

`pd.merge(adf, bdf, how='right', on='x1')`
Join matching rows from adf to bdf.

x1	x2	x3
A	1	T
B	2	F

`pd.merge(adf, bdf, how='inner', on='x1')`
Join data. Retain only rows in both sets.

x1	x2	x3
A	1	T
B	2	F
C	3	NaN
D	NaN	T

`pd.merge(adf, bdf, how='outer', on='x1')`
Join data. Retain all values, all rows.

Filtering Joins

x1	x2
A	1
B	2

`adf[adf.x1.isin(bdf.x1)]`
All rows in adf that have a match in bdf.

x1	x2
C	3

`adf[~adf.x1.isin(bdf.x1)]`
All rows in adf that do not have a match in bdf.

ydf

x1	x2
A	1
B	2
C	3

+

zdf

x1	x2
B	2
C	3
D	4

=

Set-like Operations

x1	x2
B	2
C	3

`pd.merge(ydf, zdf)`
Rows that appear in both ydf and zdf (Intersection).

x1	x2
A	1
B	2
C	3
D	4

`pd.merge(ydf, zdf, how='outer')`
Rows that appear in either or both ydf and zdf (Union).

x1	x2
A	1

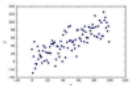
```
pd.merge(ydf, zdf, how='outer',
         indicator=True)
.query('_merge == "left_only"')
.drop(columns=['_merge'])
```
Rows that appear in ydf but not zdf (Setdiff).

Plotting

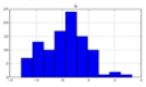
df.plot()
Plot a line graph for the DataFrame.



df.plot.scatter(x='w', y='h')
Plot a scatter graph of the DataFrame.



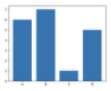
df.plot.hist()
Plot a histogram of the DataFrame.



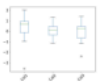
df.plot.pie()
Plot a pie chart of the DataFrame.



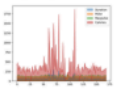
df.plot.bar()
Plot a line graph for the DataFrame.



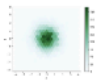
df.plot.boxplot()
Plot a scatter graph of the DataFrame.



df.plot.area()
Plot an area graph of the DataFrame.



df.plot.hexbin()
Plot a hexbin graph of the DataFrame.



df.plot(subplots=True)
Separate into different graphs for each column in the DataFrame.

df.plot(title="Graph of A against B")
Sets the title of the graph.

df.plot(cumulative=True)
Creates a cumulative plot

df.plot(bins=30)
Set the number of bins into which data is grouped (histograms)

df.plot(stacked=True)
Stacks the data for the columns on top of each other. (bar, barh and area only)

df.plot(alpha=0.5)
Sets the transparency of the plot to 50%.

df.plot(subplots=True, title=['col1', 'col2', 'col3'])
Arguments can be combined for more flexibility when graphing, this would plot a separate line graph for of column of a 3-columned DataFrame. The first string in the list of titles applies to the graph of the left-most column.

Changing Type

pd.to_numeric(data)
Convert non-numeric types to numeric.

pd.to_datetime(data)
Convert non-datetime types to datetime type

pd.to_timedelta(data)
Convert non- timedelta types to timedelta

df.astype(type)
Convert data to (almost) any given type including categorical

df.infer_objects()
Attempts to infer a better type for object type data.

df.convert_dtypes()
Convert columns to best possible dtypes

Datetime

With a Series containing data of type datetime, the dt accessor is used to get various components of the datetime values:

s.dt.year
Extract the year

s.dt.month
Extract the month as an integer.

s.dt.day
Extract the day (int) from the date.

s.dt.quarter
Find which quarter the date lies in.

s.dt.hour
Extract the hour.

s.dt.minute
Extract the minute.

s.dt.second
Extract the second.

Mapping

Apply a mapping to every element in a DataFrame or Series, useful for recategorizing or transforming data.

s.map(lambda x: 2*x)
Returns a copy of the series where every entry is doubled

df.apply(lambda s: s.max() - s.min(), axis=1)
Returns a Series with the difference of the maximum and minimum values of each row of the DataFrame

Series String Operations

Similar to python string operations, except these are vectorized to apply to the entire Series efficiently.

s.str.count(pattern)
Returns a series with the integer counts in each element.

s.str.get(index)
Returns a series with the data at the given index for each element.

s.str.join(sep)
Returns a series where each element has been concatenated.

s.str.title()
Converts the first character of each word to be a capital.

s.str.len()
Returns a series with the lengths of each element.

s.str.cat()
Concatenate elements into a single string

s.str.partition(sep)
Splits the string on the first instance of the separator

s.str.slice(start, stop, step)
Slices each string

s.str.replace(pat, rep)
Use regex to replace patterns in each string.

s.str.isalnum()
Checks whether each element is alpha-numeric

Input/Output

Common file types for data input include CSV, JSON, HTML which are human-readable, while the common output types are usually more optimized for performance and scalability such as feather, parquet and HDF.

df = pd.read_csv(filepath)
Read data from csv file

df = pd.read_html(filepath)
Read data from html file

df = pd.read_excel(filepath)
Read data from xls (and related) files

df = pd.read_sql(filepath)
Read data from sql file

pd.read_clipboard()
Read text from clipboard

df.to_parquet(filepath)
Write data to parquet file

df.to_feather(filepath)
Write data to feather file

df.to_hdf(filepath)
Write data to HDF file

df.to_clipboard()
Copy object to the system clipboard

Frequently Used Options

Pandas offers some 'options' to globally control how Pandas behaves, display etc. Options can be queried and set via:

pd.options.*option_name* (where *option_name* is the name of an option). For example:

pd.options.display.max_rows = 20
Set the **display.max_rows** option to 20.

Functions

get_option(option)
Fetch the value of the given option.

set_option(option)
Set the value of the given option.

reset_option(options)
Reset the values of all given options to default settings.

describe_option(options)
Print descriptions of given options.

option_context(options)
Execute code with temporary option settings that revert to prior settings after execution.

Display options

display.max_rows
The maximum number of rows displayed in pretty-print.

display.max_columns
The maximum number of columns displayed in pretty-print.

display.expand_frame_repr
Controls whether the DataFrame representation stretches across pages.

display.large_repr
Controls whether a DataFrame that exceeds maximum rows/columns is truncated or summarized

display.precision
The output display precision in decimal places.

display.max_colwidth
The maximum width of columns, longer cells will be truncated.

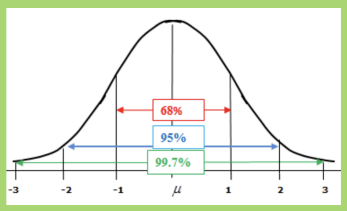
display.max_info_columns
The maximum number of columns displayed after calling **info()**.

display.chop_threshold
Sets the rounding threshold to zero when displaying a Series/DataFrame.

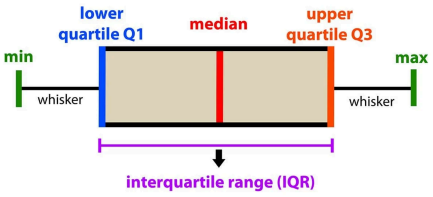
display.colheader_justify
Controls how column headers are justified.

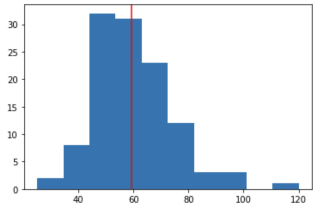
Descriptive Statistics

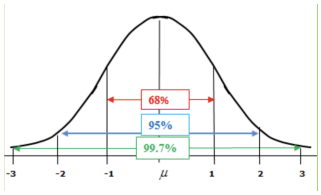
KEY VOCABULARY

`.describe()`	Shows descriptive statistics (mean, median, mode, max, min, quartiles, standard deviation) of all attributes in the dataset.
Mean (Average)	The numeric sum, divided by the total number of values in a set
Median	The middle element in a sorted set of values (numeric or ordinal)
Mode	The most frequently repeated element in a set of values.
Standard Deviation	The measure of how far each observed value is from the mean (numeric)
Quartiles	One quarter of the data (first is 0 - 25%, second is 25 - 50%, etc)
Empirical Rule 	In a normal distribution, ■ Approximately 68% of the data will fall within 1 standard deviation of the mean. ■ Approximately 95% will fall within 2 standard deviations of the mean. ■ Approximately 99.7% will fall within 3 standard deviations of the mean.

Visualizations

Boxplot:
`plt.boxplot(attribute1)`
`plt.show()`
	

Histogram:
`plt.hist(coaster_df['Top_Speed'])`
`plt.axvline(speeds.mean(), color = "red")`
`plt.show()`
	

Bell Curve:
`mu, std = norm.fit(attribute1)`
`xmin, xmax = plt.xlim()`
`x = np.linspace(xmin, xmax, 100)`
`p = norm.pdf(x, mu, std)`
`plt.plot(x, p, 'k', linewidth=2)`
`plt.show()`
	

Statistics - Using Python

STEPS - PYTHON COMMANDS

Import Libraries	`import pandas as pd`
Read in the data	`my_df = pd.read_csv('./NAME OF FILE.csv')`
Conduct data cleaning as needed	
Find the total number of entries	`attribute1 = my_df['Attribute 1']` `attribute1.count()`
Find the maximum and minimum values	`max = attribute1.max()` `min = attribute1.min()`
Calculate mean	`mean = attribute1.mean()`
Calculate median	`median = attribute1.median()`
Calculate mode (categorical only!)	`mode = attribute1.mode()`
Calculate standard deviation	`stdev = attribute1.std()`

KEY VOCABULARY

`.describe()`	Shows the descriptive statistics (mean, median, mode, maximum, minimum, quartiles, standard deviation) of all attributes in the dataset.

EXAMPLE

```
import pandas as pd
import matplotlib.pyplot as plt
coaster_df = pd.read_csv('./roller_coaster.csv')
coaster_df.dropna(inplace = True, subset = ['Top_Speed'])
speeds = coaster_df['Top_Speed']
mean = speeds.mean()
median = speeds.median()
mode = speeds.mode()
stdev = speeds.std()
print("The mean top speed is", mean)
print("The median top speed is", median)
print("The most common top speed is", mode)
print("The standard top speed of depths is", stdev)
```

Practice Assignment 🔑

Creating Visualizations

STEPS - PYTHON COMMANDS

Import Libraries	`import pandas as pd` `import matplotlib.pyplot as plt`

Read in the data	`my_df = pd.read_csv('./FILE NAME.csv')`
	`attribute1 = my_df['attribute 1']`
	`attribute2 = my_df['attribute 2']`

Create Plots	
Boxplots `plt.boxplot(attribute1)`	Histograms `plt.hist(attribute1)`

Scatterplots
`plt.scatter(attribute1, attribute2)`
	Line Graphs
	`data= data.sort_values(by=['x attribute'])`
	`plt.plot(attribute1, attribute2)`

Bar Charts `attribute1.value_counts().plot.bar()`	Pie Charts `attribute1.value_counts().plot(kind='pie')`
Titles & Labels	
Scatterplots, boxplots, histograms, line, etc.	Bar charts, pie charts, etc

`plt.title("Title with your Initials")`
`plt.xlabel('Title for attribute1')`
`plt.ylabel('Title for attribute2')`
	`plt.title("Title with your Initials")`
	`plt.legend(['Category 1', 'Category2'])`

See documentation for other parameters. For example: `color = "some color"`	
Show Plots	
`plt.show()`	

EXAMPLE

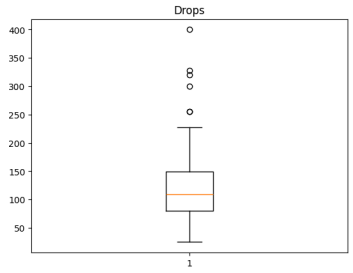
```
import pandas as pd
import matplotlib.pyplot as plt
coaster_df = pd.read_csv('./roller_coaster.csv')

heights = coaster_df['Max_Height']
speeds = coaster_df['Top_Speed']
types = coaster_df['Type']
design = coaster_df['Design']
drop = coaster_df['Drop']
```

Boxplot

One Numeric Attribute

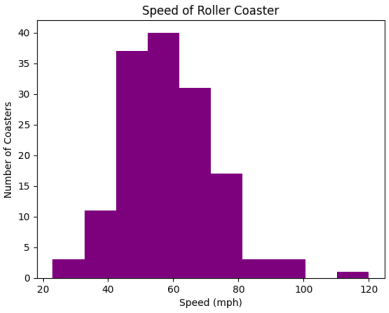
```
plt.boxplot(drop)
plt.title("Drops")
plt.show()
```



Histogram

One Numeric Attribute

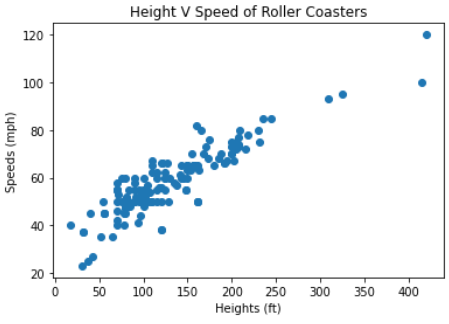
```
plt.hist(speeds, c = 'purple')
plt.title('Speed of Roller Coaster')
plt.xlabel('Speed (mph)')
plt.ylabel("Number of Coasters")
plt.show()
```



Scatterplot

Two Numeric Attributes

```
plt.scatter(heights, speeds)
plt.title("Height V Speed of Roller Coasters")
plt.xlabel('Heights (ft)')
plt.ylabel('Speeds (mph)')
plt.show()
```

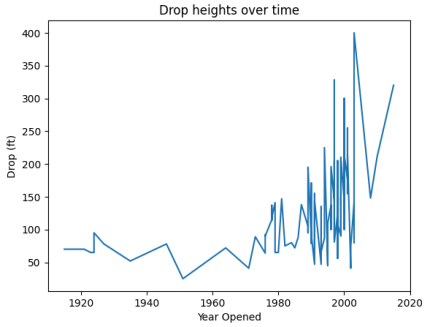


Line Graph

Two Numeric Attributes (One Continuous)

```
coaster_data = coaster_data.sort_values(by=['Year_Opened'])

year = coaster_data['Year_Opened']
drop = coaster_data['Drop']
plt.plot(year, drop)
plt.title("Drop heights over time")
plt.xlabel("Year Opened")
plt.ylabel("Drop (ft)")
plt.show()
```

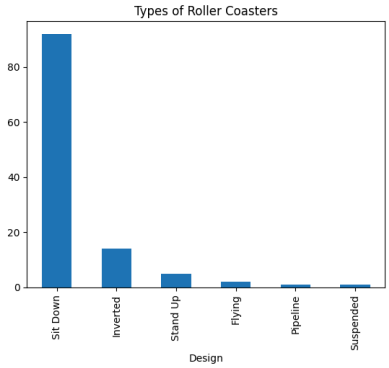


Bar Chart

One Categorical Attribute

```
design.value_counts().plot.bar(title="Types of Roller Coasters"))

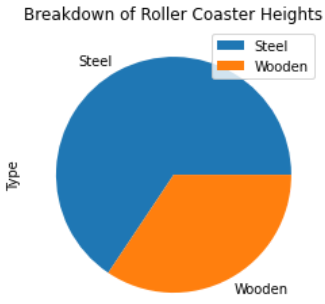
plt.show()
```



Pie Chart

One Categorical Attribute

```
types.value_counts().plot(kind='pie')
plt.title("Breakdown of Roller Coaster Heights")
plt.legend(['Steel', 'Wooden'])
plt.show()
```



[Practice Assignment](#) 🔑

Advanced Visualizations

STEPS - PYTHON COMMANDS

Import Libraries	`import pandas as pd` `import matplotlib.pyplot as plt`
Read in the data	`my_df = pd.read_csv('./FILE NAME.csv')`
Create Subsets	`cat1 = my_df[my_df['attribute']== 'category 1']` `cat2 = my_df[my_df['attribute']== 'category 2']`
Create Plots	
Side-by-Side boxplot	`ax = my_df.boxplot(column = 'attribute1', by='attribute2')`
Overlaid Histogram	`plt.hist(cat1['Attribute 1'], alpha = 0.5)` `plt.hist(cat2['Attribute 1'], alpha = 0.5)`

Side-by-Side Pie	`fig = plt.figure()`
	`labels = my_df['attribute'].unique()`
	`ax1 = fig.add_axes([0, 0, .5, .5], aspect=1)`
	`ax1.pie(cat1['attribute'].value_counts(), labels=labels)`
	`ax2 = fig.add_axes([.5, .0, .5, .5], aspect=1)`
	`ax2.pie(cat2['attribute'].value_counts(), labels=labels)`
	`ax1.set_title('Title 1')`
	`ax2.set_title('Title 2')`

Scatter / Line *(use `.plot()` for line)*	`plt.scatter(cat1['Attribute 1'], cat1['Attribute 2'])` `plt.scatter(cat2['Attribute 1'], cat2['Attribute 2'])`

Stacked Bar
(for a grouped bar, delete `stacked = True`)
	`labels = data['Attribute 1'].unique()`
	`count1 = cat1['Attribute 1'].value_counts()`
	`count2 = cat2['Attribute 1'].value_counts()`
	`df = pd.DataFrame({'cat1': count1, 'cat2':count2},`
	`index = labels)`
	`df.plot.bar(stacked = True, title = 'Title')`

See documentation to add titles and change parameters	
Show Plots	
`plt.show()`	

EXAMPLE

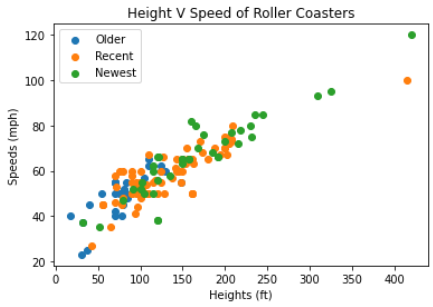
```
import pandas as pd
import matplotlib.pyplot as plt
coaster_df = pd.read_csv('./roller_coaster.csv')
coaster_df = coaster_df.sort_values(by=['Year_Opened'])

just_old = coaster_df[coaster_df['Age_Group']== 'older']
just_recent = coaster_df[coaster_df['Age_Group']== 'recent']
just_new = coaster_df[coaster_df['Age_Group']== 'newest']
```

Scatterplot

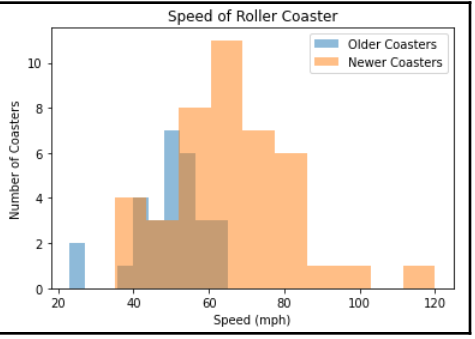
```
plt.scatter(just_old['Max_Height'], just_old['Top_Speed'])
plt.scatter(just_recent['Max_Height'], just_recent['Top_Speed'])
plt.scatter(just_new['Max_Height'], just_new['Top_Speed'])

plt.title("Height V Speed of Roller Coasters")
plt.xlabel('Heights (ft)')
plt.ylabel('Speeds (mph)')
plt.legend(['Older', 'Recent', 'Newest'])
plt.show()
```



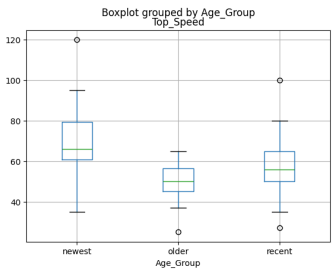
Histogram

```
plt.hist(just_old['Top_Speed'], alpha = 0.5)
plt.hist(just_new['Top_Speed', alpha = 0.5)
plt.legend(['Older Coasters', 'Newer Coasters'])
plt.title('Speed of Roller Coaster')
plt.xlabel('Speed (mph)')
plt.ylabel("Number of Coasters")
plt.show()
```



Box Plots

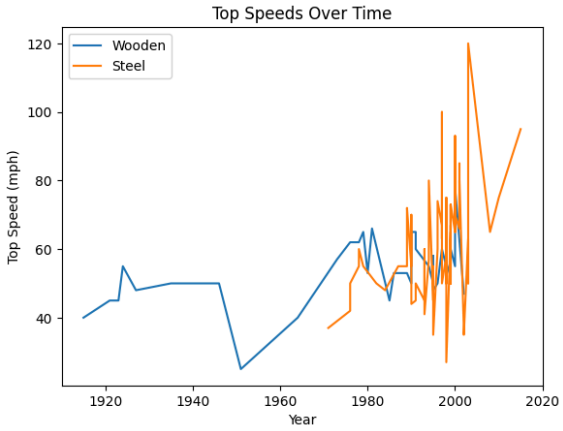
```
ax = coaster_data.boxplot(column = 'Top_Speed', by='Age_Group')
plt.show()
```



Line Graph

```
just_wooden = coaster_data[coaster_data['Type']== "Wooden"]
just_steel = coaster_data[coaster_data['Type']== "Steel"]
plt.plot(just_wooden['Year_Opened'], just_wooden['Top_Speed'])
plt.plot(just_steel['Year_Opened'], just_steel['Top_Speed'])

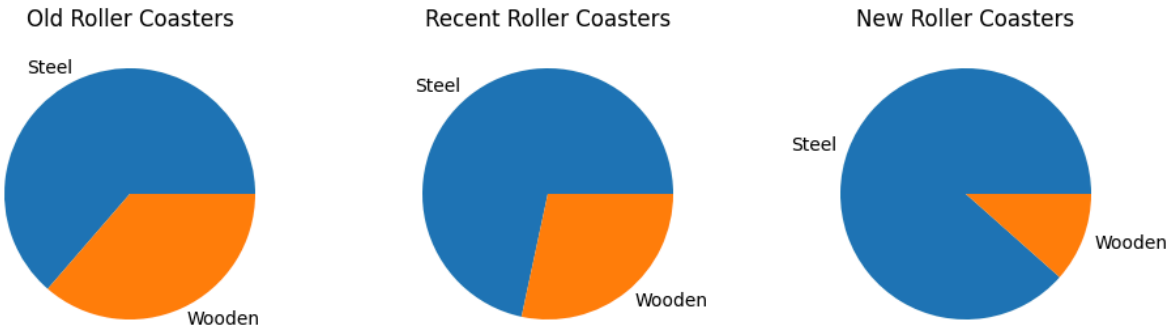
plt.title("Top Speeds Over Time")
plt.legend(['Wooden', 'Steel'])
plt.xlabel("Year")
plt.ylabel("Top Speed (mph)")
plt.show()
```



Pie Charts

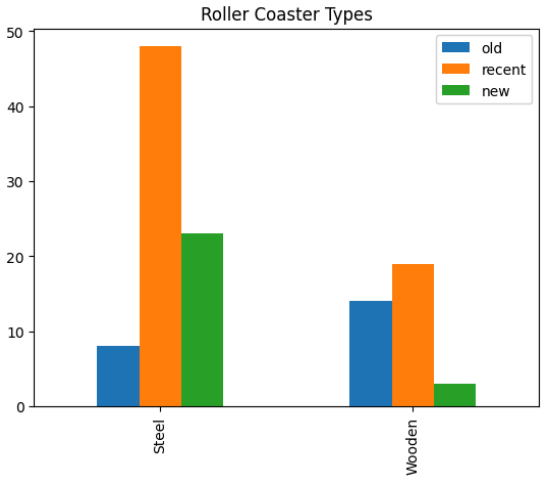
```
fig = plt.figure()
labels = coaster_data['Type'].unique()

ax1 = fig.add_axes([0, 0, .5, .5], aspect=1)
ax1.pie(just_old['Type'].value_counts(), labels=labels)
ax2 = fig.add_axes([.5, .0, .5, .5], aspect=1)
ax2.pie(just_recent['Type'].value_counts(), labels=labels)
ax3 = fig.add_axes([1, .0, .5, .5], aspect=1)
ax3.pie(just_new['Type'].value_counts(), labels=labels)
ax1.set_title('Old Roller Coasters')
ax2.set_title('Recent Roller Coasters')
ax2.set_title('New Roller Coasters')
plt.show()
```



Grouped Bar

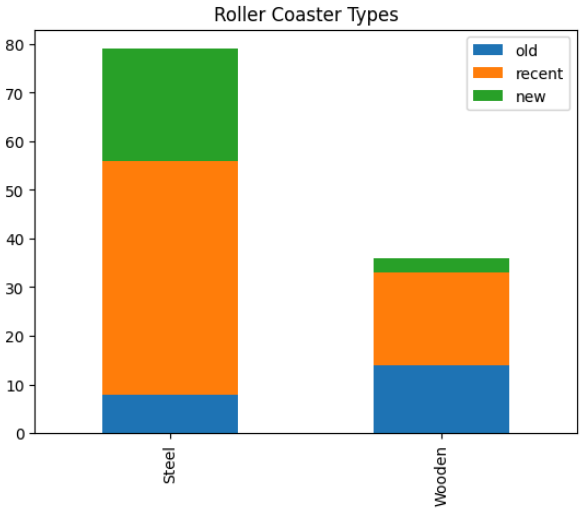
```
type = coaster_data['Type'].unique()
old = just_old['Type'].value_counts()
recent = just_recent['Type'].value_counts()
new = just_new['Type'].value_counts()
df = pd.DataFrame({'old': old, 'recent':recent, 'new':new}, index = type)
df.plot.bar(title = "Roller Coaster Types")
```



Stacked Bar

```
type = coaster_data['Type'].unique()
old = just_old['Type'].value_counts()
recent = just_recent['Type'].value_counts()
new = just_new['Type'].value_counts()

df = pd.DataFrame({'old': old, 'recent':recent, 'new':new}, index = type)
df.plot.bar(stacked = True, title = "Roller Coaster Types")
```



Practice Assignment 🔑

matplotlib Cheat sheet

Version 3.7.4

Quick start

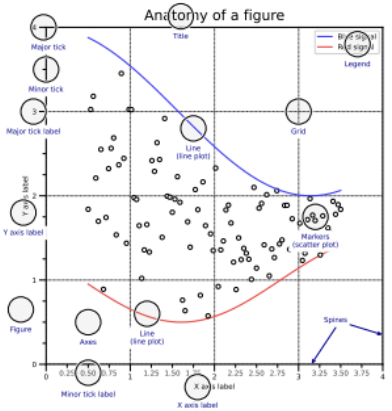
```
import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt


X = np.linspace(0, 2*np.pi, 100)
Y = np.cos(X)

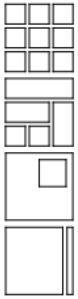
fig, ax = plt.subplots()
ax.plot(X, Y, color='green')

fig.savefig("figure.pdf")
plt.show()
```

Anatomy of a figure



Subplots layout



`subplot[s](rows, cols, …)`
`fig, axs = plt.subplots(3, 3)`

`G = gridspec(rows,cols, …)`
`ax = G[0, :]`

`ax.inset_axes(extent)`

`d=make_axes_locatable(ax)`
`ax = d.new_horizontal('10%')`

Getting help

- matplotlib.org
- github.com/matplotlib/matplotlib/issues
- discourse.matplotlib.org
- stackoverflow.com/questions/tagged/matplotlib
- https://gitter.im/matplotlib/matplotlib
- twitter.com/matplotlib
- Matplotlib users mailing list

Basic plots



`plot([X], Y, [fmt], …)`
X, Y, fmt, color, marker, linestyle



`scatter(X, Y, …)`
X, Y, [s]izes, [c]olors, marker, cmap



`bar[h](x, height, …)`
x, height, width, bottom, align, color



`imshow(Z, …)`
Z, cmap, interpolation, extent, origin



`contour[f]([X], [Y], Z, …)`
X, Y, Z, levels, colors, extent, origin



`pcolormesh([X], [Y], Z, …)`
X, Y, Z, vmin, vmax, cmap



`quiver([X], [Y], U, V, …)`
X, Y, U, V, C, units, angles



`pie(X, …)`
Z, explode, labels, colors, radius



`text(x, y, text, …)`
x, y, text, va, ha, size, weight, transform



`fill[_between][x](…)`
X, Y1, Y2, color, where

Advanced plots



`step(X, Y, [fmt], …)`
X, Y, fmt, color, marker, where



`boxplot(X, …)`
X, notch, sym, bootstrap, widths



`errorbar(X,Y,xerr,yerr, …)`
X, Y, xerr, yerr, fmt



`hist(X, bins, …)`
X, bins, range, density, weights



`violinplot(D, …)`
D, positions, widths, vert



`barbs([X], [Y], U, V, …)`
X, Y, U, V, C, length, pivot, sizes



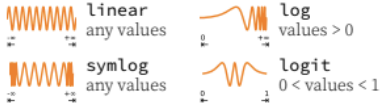
`eventplot(positions, …)`
positions, orientation, lineoffsets



`hexbin(X, Y, C, …)`
X, Y, C, gridsize, bins

Scales

`ax.set_[xy]scale(scale, …)`



Projections

`subplot(…, projection=p)`



p='polar'



p='3d'

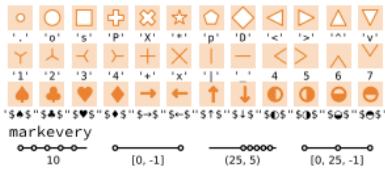


p=ccrs.Orthographic()
import cartopy.crs as ccrs

Lines



Markers

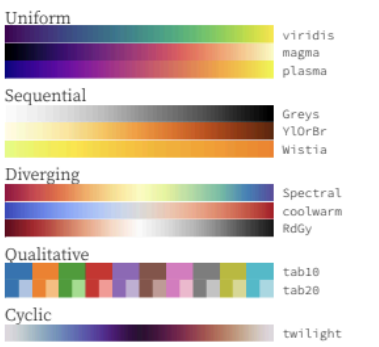


Colors



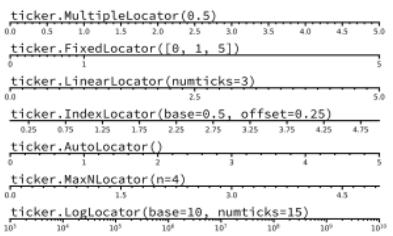
Colormaps

`plt.get_cmap(name)`



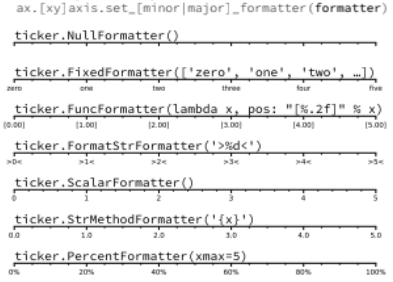
Tick locators

```
from matplotlib import ticker
ax.[xy]axis.set_[minor|major]_locator(locator)
```



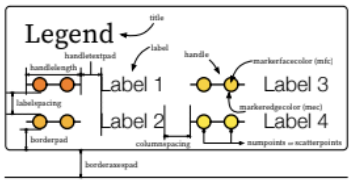
Tick formatters

```
from matplotlib import ticker
ax.[xy]axis.set_[minor|major]_formatter(formatter)
```

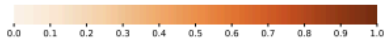


Ornaments

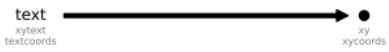
`ax.legend(…)`
handles, labels, loc, title, frameon



`ax.colorbar(…)`
mappable, ax, cax, orientation



`ax.annotate(…)`
text, xy, xytext, xycoords, textcoords, arrowprops



Event handling

```
fig, ax = plt.subplots()
def on_click(event):
  print(event)
fig.canvas.mpl_connect(
  'button_press_event', on_click)
```

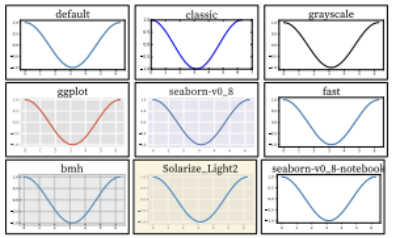
Animation

```
import matplotlib.animation as mpla

T = np.linspace(0, 2*np.pi, 100)
S = np.sin(T)
line, = plt.plot(T, S)
def animate(i):
    line.set_ydata(np.sin(T+i/50))
anim = mpla.FuncAnimation(
    plt.gcf(), animate, interval=5)
plt.show()
```

Styles

`plt.style.use(style)`



Quick reminder

```
ax.grid()
ax.set_[xy]lim(vmin, vmax)
ax.set_[xy]label(label)
ax.set_[xy]ticks(ticks, [labels])
ax.set_[xy]ticklabels(labels)
ax.set_title(title)
ax.tick_params(width=10, …)
ax.set_axis_[on|off]()

fig.suptitle(title)
fig.tight_layout()
plt.gcf(), plt.gca()
mpl.rc('axes', linewidth=1, …)
[fig|ax].patch.set_alpha(0)
text=r'$\frac{-e^{i\pi}}{2^n}$'
```

Keyboard shortcuts

ctrl + s	Save	ctrl + w	Close plot
r	Reset view	f	Fullscreen 0/1
f	View forward	b	View back
p	Pan view	o	Zoom to rect
x	X pan/zoom	y	Y pan/zoom
g	Minor grid 0/1	G	Major grid 0/1
l	X axis log/linear	L	Y axis log/linear

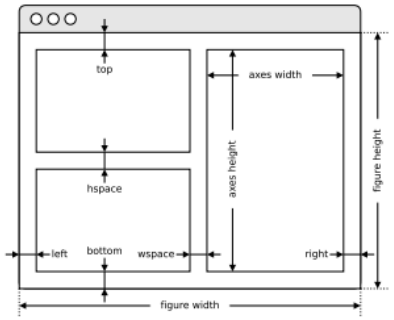
Ten simple rules

1. Know your audience
2. Identify your message
3. Adapt the figure
4. Captions are not optional
5. Do not trust the defaults
6. Use color effectively
7. Do not mislead the reader
8. Avoid "chartjunk"
9. Message trumps beauty
10. Get the right tool

Axes adjustments

API

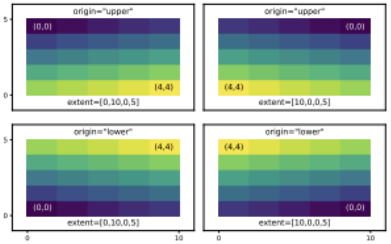
plt.**subplots_adjust**(...)



Extent & origin

API

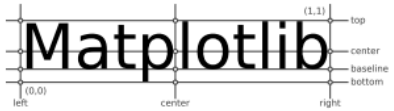
ax.**imshow**(extent=..., origin=...)



Text alignments

API

ax.**text**(..., ha=... , va=..., ...)



Text parameters

API

ax.**text**(..., family=..., size=..., weight=...)
ax.**text**(..., fontproperties=...)

The quick brown fox — xx-large (1.73)
The quick brown fox — x-large (1.44)
The quick brown fox — large (1.20)
The quick brown fox — medium (1.00)
The quick brown fox — small (0.83)
The quick brown fox — x-small (0.69)
The quick brown fox — xx-small (0.58)

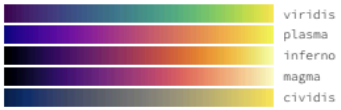
The quick brown fox jumps over the lazy dog — black (900)
The quick brown fox jumps over the lazy dog — bold (700)
The quick brown fox jumps over the lazy dog — semibold (600)
The quick brown fox jumps over the lazy dog — normal (400)
The quick brown fox jumps over the lazy dog — ultralight (100)

The quick brown fox jumps over the lazy dog — monospace
The quick brown fox jumps over the lazy dog — serif
The quick brown fox jumps over the lazy dog — sans
The quick brown fox jumps over the lazy dog — cursive

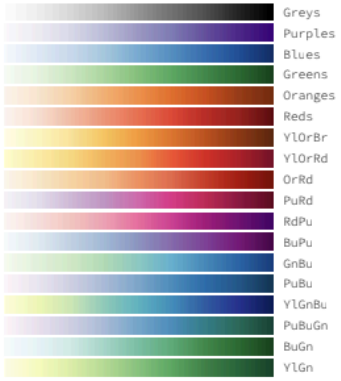
The quick brown fox jumps over the lazy dog — italic
The quick brown fox jumps over the lazy dog — normal

THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG — small-caps
The quick brown fox jumps over the lazy dog — normal

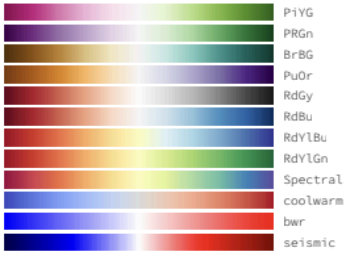
Uniform colormaps



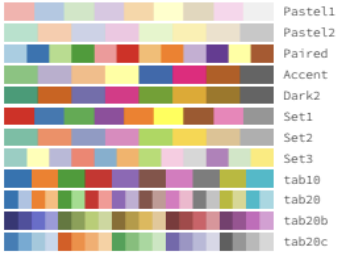
Sequential colormaps



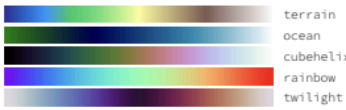
Diverging colormaps



Qualitative colormaps



Miscellaneous colormaps



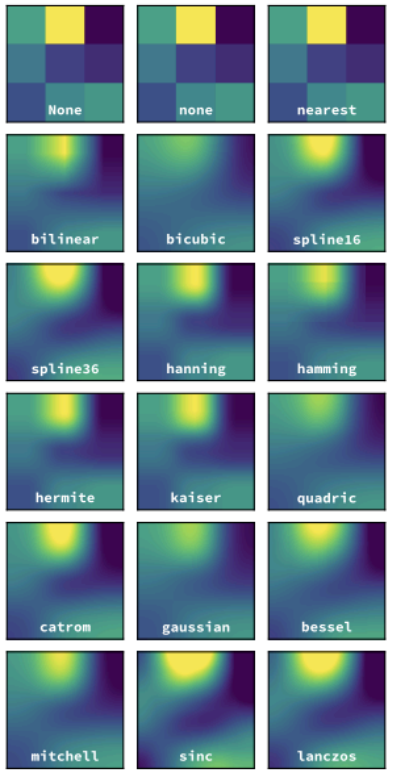
Color names

API

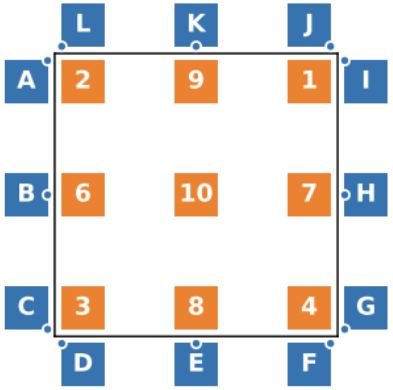


Image interpolation

API



Legend placement



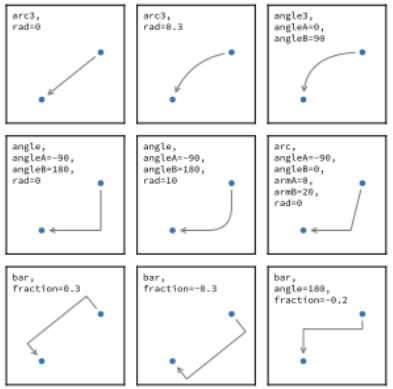
ax.**legend**(loc="string", bbox_to_anchor=(x, y))

2: upper left — 9: upper center — 1: upper right
6: center left — 10: center — 7: center right
3: lower left — 8: lower center — 4: lower right

A: upper right / (-0.1, 0.9) — B: center right / (-0.1, 0.5)
C: lower right / (-0.1, 0.1) — D: upper left / (0.1, -0.1)
E: upper center / (0.5, -0.1) — F: upper right / (0.9, -0.1)
G: lower left / (1.1, 0.1) — H: center left / (1.1, 0.5)
I: upper left / (1.1, 0.9) — J: lower right / (0.9, 1.1)
K: lower center / (0.5, 1.1) — L: lower left / (0.1, 1.1)

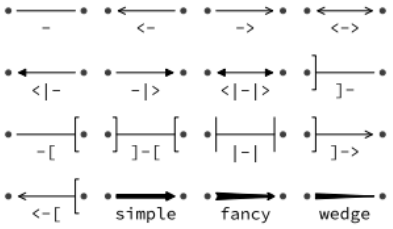
Annotation connection styles

API



Annotation arrow styles

API



How do I ...

... resize a figure?
→ fig.set_size_inches(w, h)
... save a figure?
→ fig.savefig("figure.pdf")
... save a transparent figure?
→ fig.savefig("figure.pdf", transparent=True)
... clear a figure/an axes?
→ fig.clear() → ax.clear()
... close all figures?
→ plt.close("all")
... remove ticks?
→ ax.set_[xy]ticks([])
... remove tick labels ?
→ ax.set_[xy]ticklabels([])
... rotate tick labels ?
→ ax.tick_params(axis="x", rotation=90)
... hide top spine?
→ ax.spines['top'].set_visible(False)
... hide legend border?
→ ax.legend(frameon=False)
... show error as shaded region?
→ ax.fill_between(X, Y+error, Y-error)
... draw a rectangle?
→ ax.add_patch(plt.Rectangle((0, 0), 1, 1)
... draw a vertical line?
→ ax.axvline(x=0.5)
... draw outside frame?
→ ax.plot(..., clip_on=False)
... use transparency?
→ ax.plot(..., alpha=0.25)
... convert an RGB image into a gray image?
→ gray = 0.2989*R + 0.5870*G + 0.1140*B
... set figure background color?
→ fig.patch.set_facecolor("grey")
... get a reversed colormap?
→ plt.get_cmap("viridis_r")
... get a discrete colormap?
→ plt.get_cmap("viridis", 10)
... show a figure for one second?
→ fig.show(block=False), time.sleep(1)

Performance tips

```
scatter(X, Y)                                  slow
plot(X, Y, marker="o", ls="")                  fast

for i in range(n): plot(i, X[i], "o")          slow
plot(X, marker="o", ls="")                     fast

cla(); imshow(…); canvas.draw()                slow
im.set_data(…); canvas.draw()                  fast
```

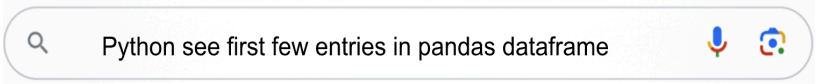
Beyond Matplotlib

Seaborn: Statistical data visualization
Cartopy: Geospatial data processing
yt: Volumetric data visualization
mpld3: Bringing Matplotlib to the browser
Datashader: Large data processing pipeline
plotnine: A grammar of graphics for Python

Matplotlib Cheatsheets
Copyright (c) 2021 Matplotlib Development Team
Released under a CC-BY 4.0 International License

NUMFOCUS
OPEN CODE = BETTER SCIENCE

Documentation

How to find commands	Google: “Language” + what you want to do
	
Option 1: Google AI	Google might give an AI overview. This is a time where using AI to help you code is allowed. Read the examples and apply to what you are doing.
Option 2: Help Sites	If AI is not there or doesn’t work, find help sites & examples

Understanding Commands

How to find documentation	Google: “Language” + “Library” + “Command” + “Documentation”
	
Option 1: Google AI	Google might give you an AI overview that you can use.
Option 2: Documentation	If AI is not there or doesn’t work, find the documentation for the library. Pandas https://pandas.pydata.org › docs › reference › api › pand... pandas.DataFrame.head — pandas 2.3.0 documentation

Documentation Part 1: Overview	Part 2: Examples

pandas.DataFrame.head
DataFrame.head(*n=5*) [source]
If it has an “=” that means you don’t have to put in that parameter and it will default to what is in the documentation.
Return the first *n* rows.
This function returns the first *n* rows for the object based on position. It is useful for quickly testing if your object has the right type of data in it. What the command does.
For negative values of *n*, this function returns all rows except the last /*n*/ rows, equivalent to `df[:n]`.
If n is larger than the number of rows, this function returns all rows.
Parameters: Listed inside the parentheses. These are the things you can change.
n : *int, default 5*
Number of rows to select.
Data type you can put in
Returns:
same type as caller If it does not have an “=”, you must put something in.
The first *n* rows of the caller object.
	Scroll down. Often, reading the examples first is easier!
	Examples
	`>>> df = pd.DataFrame({'animal': ['alligator', 'bee', 'falcon', 'lion', ... 'monkey', 'parrot', 'shark', 'whale', 'zebra']})`
	`>>> df`
	animal
	0 alligator
	1 bee
	2 falcon
	3 lion
	4 monkey
	5 parrot
	6 shark
	7 whale
	8 zebra
	Viewing the first 5 lines
	`>>> df.head()`
	animal
	0 alligator
	1 bee
	2 falcon
	3 lion
	4 monkey
	Viewing the first *n* lines (three in this case)
	`>>> df.head(3)`
	animal
	0 alligator
	1 bee
	2 falcon

Unit 3: Data Modeling and Analysis

Standard 1: Regressions (DS.9)	Not Yet	Almost	Yes
Learning Experience #1: Regressions (linear, nonlinear, exponential, logistic)			
I can **recognize** a pattern of regression and **categorize** it to the best model (all types)			
I can use the computer to **build** a regression model for given data (all types)			
I can use the computer to **graph** a regression model (all types)			
When appropriate, I can use a regression model to **predict** values (all types)			

Standard 2: Distributions (DS.9)	Not Yet	Almost	Yes
Learning Experience #1: Distributions			
I can **recognize** and **interpret** a normal distribution			
I can **evaluate** the skew of a normal distribution			
I can **identify** other types of distributions			
I can use the computer to **model** a distribution with given data			

Standard 3: Limitations (DS.9)	Not Yet	Almost	Yes
Learning Experience #1: Limitations			
I can **identify** the limitations and biases in a regression model			
I can **identify** the limitations and biases in a distribution model			
I can **identify** the limitations and biases in other models			
I can **identify** noise and overfitting in any type of model			

Standard 4: Other Models (DS.9)	Not Yet	Almost	Yes
Learning Experience #4: Other Models (Decision Trees)			
I can **interpret** a decision tree and use it to **make predictions**			
I can use the computer to **create** an appropriate decision tree for given data			
I can **compare** the effectiveness of decision trees based on features (ex. nodes)			
I can **interpret** other types of mathematical models			

Linear Regression

PYTHON STEPS

Import Libraries	`import matplotlib.pyplot as plt`
	`import pandas as pd`
	`from scipy.stats import linregress`

Read in the data	`data = pd.read_csv("./my_file_path")`
	`data = data.dropna()`
	`x = data['attribute1']`
	`y = data['attribute2']`

Run the Linear Regression	`result = linregress(x, y)`
	`slope = result.slope`
	`intercept = result.intercept`
	
	`print("Linear Regression Line: y =", slope, 'x +', intercept)`

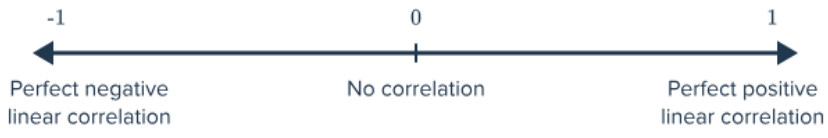
Calculate r-value	`print("R-value: ", result.rvalue)`

Plot the Linear Regression	`plt.scatter(x, y)`
	`plt.plot(x, slope*x+intercept)`
	`plt.title("TITLE")`
	`plt.xlabel("X VARIABLE")`
	`plt.ylabel("Y VARIABLE")`
	`plt.show()`

Make a Prediction	`x_val = number`
	`prediction = slope*(x_val)+intercept`
	`print("Predictor:", x_val)`
	`print("Prediction:", prediction)`

KEY VOCABULARY

r-value	A number between -1 and 1 that shows how strongly and in what direction two variables are correlated



Interpreting Predictions	If [Predictor], we can expect [Prediction]

EXAMPLE

```
from scipy.stats import linregress
import pandas as pd
import matplotlib.pyplot as plt

salaries = pd.read_csv('./Salary.csv')
x = salaries['YearsExperience']
y = salaries['Salary']

result = linregress(x, y)
slope = result.slope
intercept = result.intercept
print("Linear Regression Line: y =", slope, 'x +', intercept)
```

```
Linear Regression Line: y = 8731.94 x + 28859.53
```

```
print("R-value: ", result.rvalue)
```

```
R-value:  0.9824272546479683
```

This suggests a very strong positive relationship

```
plt.scatter(x, y)
plt.plot(x, slope*x+intercept)
plt.title("Experience & Salaries")
plt.xlabel("Years of Experience")
plt.ylabel("Salary (USD)")
plt.show()
```




```
x_val = 10
prediction = slope*(x_val)+intercept
print("Predictor:", x_val)
print("Prediction:", prediction)
```

```
Predictor: 10
Prediction: 116178.94
```

If someone has 10 years of experience, we can expect them to make approximately $116,178.94

[Practice Assignment](#) 🔑

Polynomial Regression

Use polynomial regression when data has a relationship that is not linear.

PYTHON STEPS

Import Libraries	`import numpy as np`
	`import matplotlib.pyplot as plt`
	`import pandas as pd`

Read in the data	`data = pd.read_csv("./my_file_path")`
	`data = data.dropna()`
	`x = data['attribute1']`
	`y = data['attribute2']`

Look at the data to determine the best degree	`plt.scatter(x, y)`
	`plt.title("TITLE")`
	`plt.xlabel("X VARIABLE")`
	`plt.ylabel("Y VARIABLE")`
	`plt.show()`

Run Regression	`result = np.poly1d(np.polyfit(x, y, degree_you_want)`

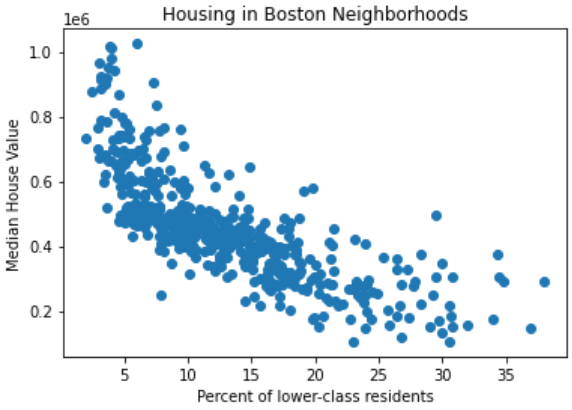
Plot the Regression	`x_vals = np.linspace(x.min(), x.max(0), 100)`
	`plt.plot(x_vals, result(x_vals))`
	`plt.scatter(x, y)`
	`plt.title("TITLE")`
	`plt.xlabel("X VARIABLE")`
	`plt.ylabel("Y VARIABLE")`
	`plt.show()`

Make a Prediction	`prediction = result(number)`
	`print("Predictor:", number)`
	`print("Prediction:", prediction)`

EXAMPLE

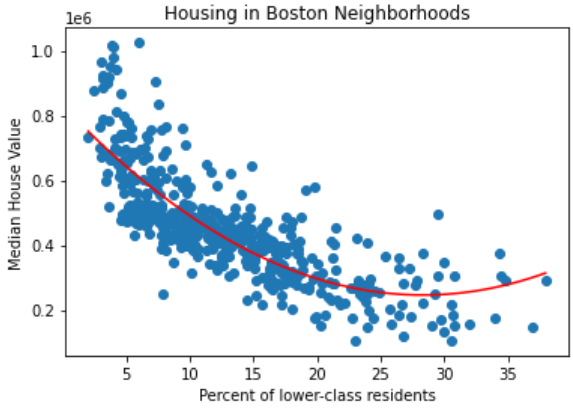
```
housing = pd.read_csv('./housing.csv')
x = housing['LSTAT']
y = salaries['MEDV']

plt.scatter(x, y)
plt.xlabel("Percent of lower-class residents")
plt.ylabel("Median House Value")
plt.title("Housing in Boston Neighborhoods")
plt.show()
```




```
house_model = np.poly1d(numpy.polyfit(x, y , 2))

house_plot = np.linspace(x.min(), x.max(), 100)
plt.plot(house_plot,house_model(house_plot),
       color = 'red')
plt.scatter(x, y)
plt.xlabel("Percent of lower-class residents")
plt.ylabel("Median House Value")
plt.title("Housing in Boston Neighborhoods")
plt.show()
```




```
med_val = house_model(25)
print("Predictor:", 25)
print("Prediction:", med_val)
```

```
Predictor: 25
Prediction: 253544.02
```

If 25% of the neighborhood is lower class, we can expect the median house price to be $253,544.02

Practice Assignment

Logistic Regression

PYTHON STEPS

Import Libraries	`import pandas as pd`
	`import matplotlib.pyplot as plt`
	`from sklearn.linear_model import LogisticRegression`
	`from scipy.special import expit`
	`import numpy as np`

Read in the data	`data = pd.read_csv("./my_file_path")`
	`data = data.dropna()`
	`x = data['attribute1'].to_numpy().reshape(-1,1)`
	`y = data['attribute2'].to_numpy()`

Run the Logistic Regression	`result = LogisticRegression(random_state=0).fit(x, y)`
	`print("Logistic Equation: p(x) = 1/(1 +`
	`exp^-(",result.intercept_[0],"+",result.coef_[0][0], "x)")`

Calc. Accuracy	`print("Accuracy: ", result.score(x, y)*100, "%")`

Plot the Logistic Regression	`x_plot = np.linspace(x.min(), x.max(), 300)`
	`to_plot = expit(result.coef_*x_plot + result.intercept_).ravel()`
	`plt.scatter(x,y)`
	`plt.plot(x_plot, to_plot)`
	`plt.title("TITLE")`
	`plt.xlabel("X VARIABLE")`
	`plt.ylabel("Y VARIABLE")`
	`plt.show()`

Make a Prediction	`x_val = number`
	`prob = expit(result.coef_*x_val+result.intercept_)`
	`print("Predictor:" , x_val)`
	`print("Prediction:" , prob)`

KEY VOCABULARY

Training Data / Testing Data	The accuracy score shows what portion of the training data (the data you used) is accurately represented with the model. If you were to use a different data set (testing data) and *didn't* get a similar accuracy score, your model is overfitted.
Logistic Regression Predictions	In logistic regression, instead of predicting *values*, you are predicting *probability*

EXAMPLE

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from scipy.special import expit
import numpy as np

advertisement = pd.read_csv('./Social_Network_Ads.csv')
advertisement = advertisement.dropna()
x = advertisement['Age'].to_numpy().reshape(-1,1)
y = advertisement['Purchased'].to_numpy()

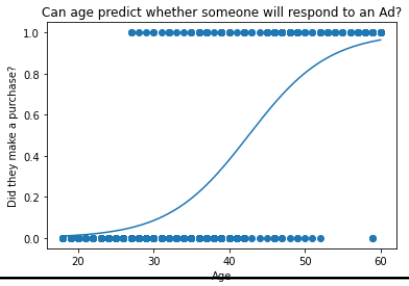
result = LogisticRegression(random_state=0).fit(x, y)
print("Logistic Equation: p(x) = 1/(1 +
exp^-(",round(result.intercept_[0],2),"+",round(result.coef_[0][0],2), "x)")
```

```
Logistic Equation: p(x) = 1/(1 + exp(-( -8.04 + 0.19 x)
```

```
print("Accuracy: ", result.score(x, y)*100, "% (for the given data)")
```

```
Accuracy:  84.0 % (for the given data)
```

```
x_plot = np.linspace(x.min(), x.max(), 300)
to_plot = expit(result.coef_*x_plot + result.intercept_).ravel()
plt.scatter(x,y)
plt.plot(x_plot, to_plot)
plt.title("Purchase Prediction")
plt.xlabel("Age")
plt.ylabel("Did they make a purchase?")
plt.show()
```




```
x_val = 40
prob = expit(result.coef_*x_val+result.intercept_)
print("Predictor:", x_val)
print("Prediction:", prob)
```

```
Predictor: 40
Prediction: 0.39174
```

If you are 40 years old, you have a 39% chance of making a purchase based on a social media ad.

[Practice Assignment](#) 🔑

Logistic Regression with Multiple Predictors

PYTHON STEPS

Import Libraries	`import pandas as pd`
	`import matplotlib.pyplot as plt`
	`from sklearn.linear_model import LogisticRegression`
	`from scipy.special import expit`
	`import numpy as np`

Read in the data	`data = pd.read_csv("./my_file_path")` `data = data.dropna()`
Separate input and output	`X = data[['attribute1','attribute2']]` `y = data[['attribute3']]`

Run the Logistic Regression	`result = LogisticRegression(random_state=0).fit(X, y)`
	`print("Logistic Equation: p(x) = 1/(1 +`
	`exp^-(",result.intercept_[0],"+",result.coef_[0][0],`
	`"*attribute1", result.coef_[0][1],"*attribute2)")`

Calculate Accuracy	`print("Accuracy: ", result.score(X, y)*100, "%")`

Make a Prediction	`first_val = number1`
	`second_val = number2`
	`prob = expit(result.coef_[0][0]*first_val+result.coef_[0][1]*`
	`second_val+result.intercept_)`
	`print("Predictor 1 (attribute):" , first_val)`
	`print("Predictor 2 (attribute):" , second_val)`
	`print("Prediction:" , prob)`

THE EQUATION

Code	`p(x) = 1/(1 + exp^-(",result.intercept_[0],"+",result.coef_[0][0],` `"*attribute1", result.coef_[0][1],"*attribute2)")`
Math	$$p(x) = \frac{1}{1+e^{-(a+b\cdot x_1+c\cdot x_2)}}$$

EXAMPLE

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from scipy.special import expit
import numpy as np

heart = pd.read_csv('./heart_disease.csv')
heart = heart.dropna()
X = heart[['cigsPerDay', 'BMI']]
y = heart['TenYearCHD']

result = LogisticRegression(random_state=0).fit(X, y)
print("Logistic Equation: p(x) = 1/(1 + exp^-(",result.intercept_[0],"+",
      result.coef_[0][0], "*cigs +", result.coef_[0][1],"*BMI)")
```

```
Logistic Equation: p(x) = 1/(1 + exp^-(-3.3001 + 0.0131 *cigs + 0.06 *BMI)
```

In mathematical notation: $$p(x) = \frac{1}{1+e^{-(-3.3001+0.0131 \cdot cigs+0.06 \cdot BMI)}}$$

```
print("Accuracy: ", result.score(x, y)*100, "% (for the given data)")
```

```
Accuracy:  84.76 % (for the given data)
```

```
cigs = 10
BMI = 20
prob = expit(result.coef_[0][0]*cigs+result.coef_[0][1]*BMI+result.intercept_)
print("Predictor 1 (cigs):" , cigs)
print("Predictor 2 (BMI):" , BMI)
print("Prediction:" , prob)
```

```
Predictor 1 (cigs): 10
Predictor 2 (BMI): 20
Prediction: 0.1139
```

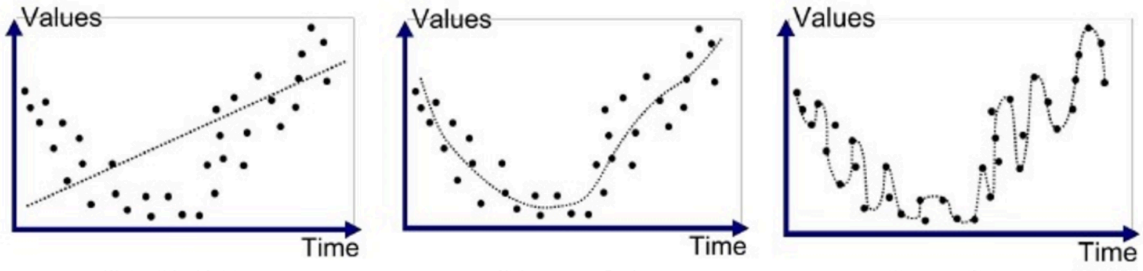
If someone smokes 10 cigarettes a day and has a BMI of 20, they have an 11.39% chance of having a heart disease.

Practice Assignment

Noise and Overfitting

KEY VOCABULARY

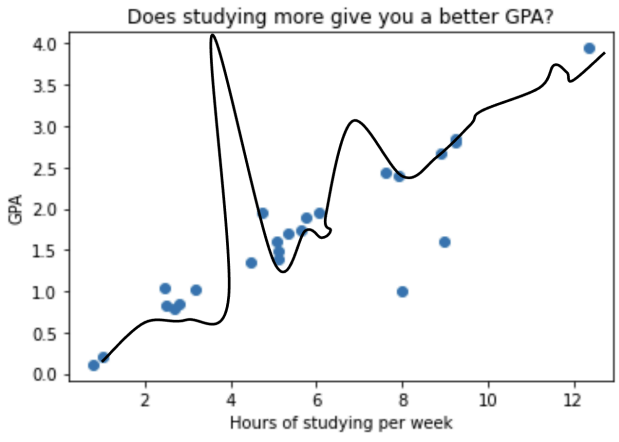
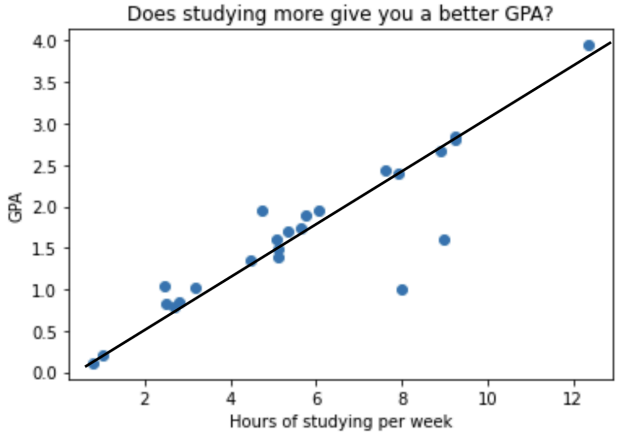
Noise	"Noise" is a word used to describe random pieces of data that make the underlying pattern less clear.		
Underfitting	**Underfitting** is when a model is too simple and leaves out some important information.	Overfitting	**Overfitting** is when useless details (i.e. noise) have too much of an effect on the model.



Underfitted | Good Fit/Robust | Overfitted

Training Dataset	The **training dataset** is the information / data used to create a model.	**Testing Dataset**	After a model is created, the **testing dataset** is <u>different</u> data that should follow the same pattern as the training dataset and is used to determine how good a model is.

RELATIONSHIP BETWEEN DATASETS AND FITTING

Testing data that shows a model is <u>overfitted</u> to the training data	**Testing data that shows a model is <u>underfitted</u> to the training data**
	
This data may fit the training data well, but does not give a good estimate with testing data	*This data does not fit either the training or testing data well.*

Normal Distribution

PYTHON STEPS

Import Libraries	`import numpy as np`
	`import matplotlib.pyplot as plt`
	`from scipy.stats import norm`
	`import pandas as pd`

Read in the data	`data = pd.read_csv("./my_file_path")`
	`data = data.dropna()`
	`attribute1 = data['attribute']`

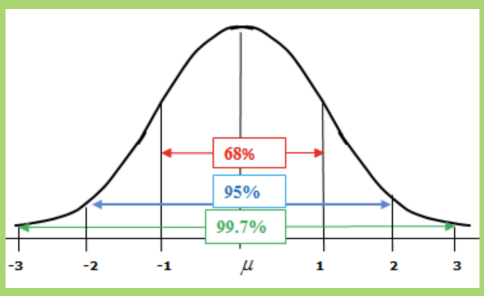
Plot a histogram	`plt.hist(attribute1, density = True)`

Calculate Normal Distribution	`mu, std = norm.fit(attribute1)`
	`xmin, xmax = plt.xlim()`
	`x = np.linspace(xmin, xmax, 100)`
	`p = norm.pdf(x, mu, std)`

Plot Normal Curve	`plt.plot(x, p, 'k', linewidth=2)`
	`plt.title("Title")`
	`plt.xlabel("Attribute")`
	`plt.show()`

Find Descriptive Statistics	`print('mean:', mu)` `print('standard deviation', std)`
Calculate a z-score	`value = number` `print('z-score of', value,':', (value-mu)/std)`

KEY VOCABULARY

Empirical Rule 	In a normal distribution, ■ Approximately 68% of the data will fall within 1 standard deviation of the mean. ■ Approximately 95% will fall within 2 standard deviations of the mean. ■ Approximately 99.7% will fall within 3 standard deviations of the mean.
Z-Score	A number that tells how many standard deviations away from the mean a given data point is

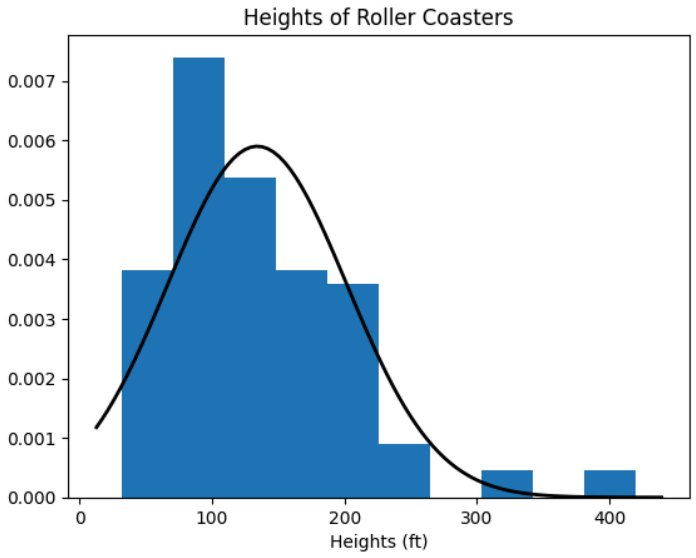
EXAMPLE

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
import pandas as pd
coaster = pd.read_csv("./roller_coaster.csv")
coaster = coaster.dropna()
heights = coaster['Max_Height']
plt.hist(heights, density = True)

mu, std = norm.fit(heights)
xmin, xmax = plt.xlim()
x = np.linspace(xmin, xmax, 100)
p = norm.pdf(x, mu, std)

plt.plot(x, p, 'k', linewidth=2)

plt.title("Heights of Roller Coasters")
plt.xlabel("Heights (ft)")
plt.show()
```




```
print('mean:', mu)
print('standard deviation', std)
value = 100
print('z-score of', value,':', (value-mu)/std)
```

```
mean: 134.0608695652174
standard deviation 67.62907619559759
z-score of 100: -0.05364
```

Practice Assignment

Decision Trees

Before beginning, categorize all of your attributes as numeric, ordinal, categorical, qualitative, or binary. Identify which attribute you are predicting.

PYTHON STEPS

Import Libraries	`from sklearn import tree`
	`import pandas as pd`
	`import matplotlib.pyplot as plt`

Read in the data	`data = pd.read_csv('./file_path.csv')` `data = data.dropna()`
Categorical	`data = pd.get_dummies(data, columns = ['categorical1',` `'categorical2', ...])`
Ordinal	`data.replace(to_replace = {'category':number ,` `'category':number,...}, inplace=True)`
Qualitative	`data= data.drop(['qual1', 'qual2', ...], axis = 1)`

Separate Predicting Variable	`bins = [number, number, number, ...]`
	`labels = ['label', 'label', 'label', ...]`
	`data['Predict'] = pd.cut(data['Predict'], bins=bins, labels=labels, right=False)`
	`data.dropna(inplace=True)`
	
	`X = data.drop('Predict',axis=1)`
	`y = data[['Predict']]`

Create the Tree	`result = tree.DecisionTreeClassifier(max_leaf_nodes=number).fit(X, y)`
	`tree.plot_tree(result, feature_names=data.drop('Predict', axis = 1).columns, class_names = result.classes_, filled=True)`
	`plt.show()`

KEY VOCABULARY

`bins`	These are the numbers that start each category of the variable you're predicting
`labels`	These are the names of each category of the variable you're predicting.

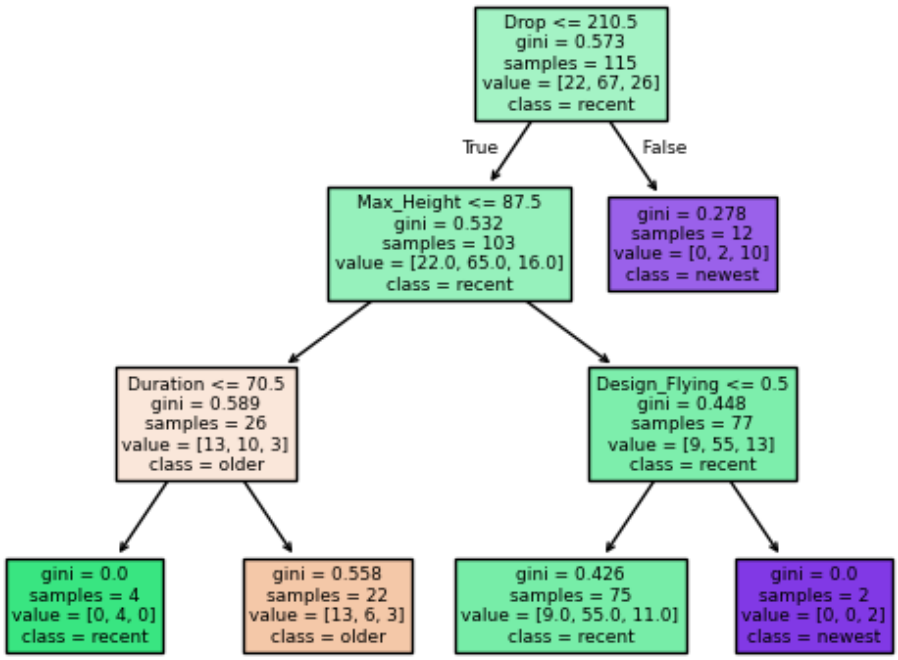
EXAMPLE 1

Predict whether the roller coaster was old, recent, or new. (Predicting = categorical)

```
from sklearn import tree
import pandas as pd
import matplotlib.pyplot as plt
coaster_data = pd.read_csv('./roller_coaster.csv')
coaster_data = coaster_data.dropna()

coaster_data = pd.get_dummies(coaster_data, columns=['Design'])
coaster_data = coaster_data.replace(to_replace = {"Wooden": 0, 'Steel': 1, 'N': 0,
     'Y': 1, 'older': 0, 'recent': 1, 'newest': 2})
coaster_data = coaster_data.drop(['Coaster', 'Park', 'City', 'Year_Opened'], axis =
      1)
bins = [0, 1, 2, 3]
labels = ['older', 'recent', 'newest']
-
coaster_data.dropna(inplace=True)

X = coaster_data.drop("Age_Group", axis = 1)
y = coaster_data[['Age_Group']]
result = tree.DecisionTreeClassifier(max_leaf_nodes=5).fit(X,y)
tree.plot_tree(result, feature_names =coaster_data.drop('Age_Group', axis =
          1).columns, class_names = result.classes_, filled = True)
plt.show()
```



This shows me that drop, height, duration, and whether it is a "flying" type are the most important attributes for determining whether the roller coaster is old, recent, or new.

For example, if a roller coaster has a drop greater than 210.5 feet, it is automatically likely new. A roller coaster with a drop of 100 feet and a height of 90 feet that is <u>not</u> a flying design is likely to be recent.

EXAMPLE 2

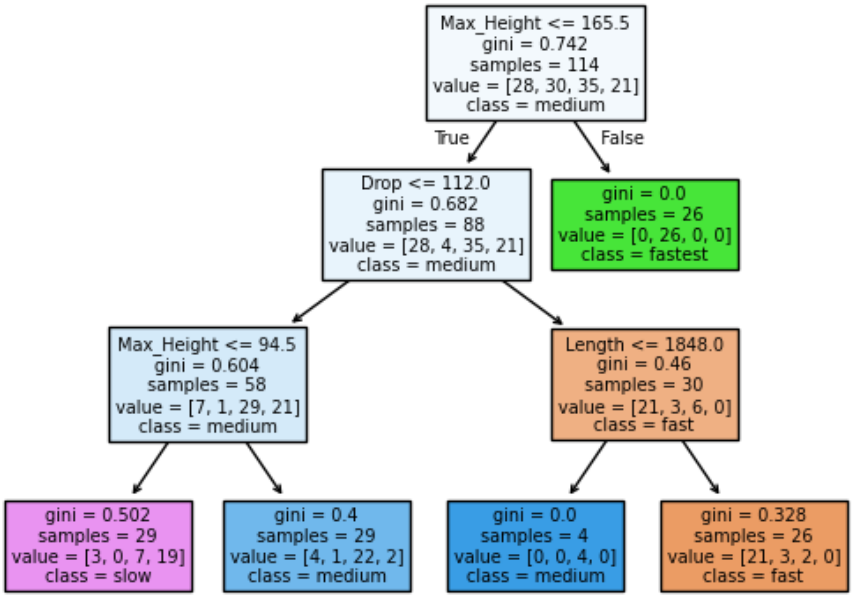
Predict whether the roller coaster was slow, medium, or fast (numeric)

```
#Begin with the same lines as example 1, up until "bins"
print(coaster_data.describe())
```

```
          Top_Speed  Max_Height  ...  Inversions  Num_of_Inversions
count   115.000000  115.000000  ...  115.000000         115.000000
mean     59.156522  134.060870  ...    0.434783           1.782609
std      14.453550   67.925047  ...    0.497898           2.412623
min      25.000000   32.000000  ...    0.000000           0.000000
25%      50.000000   92.000000  ...    0.000000           0.000000
50%      57.000000  118.000000  ...    0.000000           0.000000
75%      66.000000  161.000000  ...    1.000000           3.000000
max     120.000000  420.000000  ...    1.000000           7.000000
```

```
bins = [25, 50, 57, 66, 120]
labels = ['slow', 'medium', 'fast', 'fastest']
coaster_data['Top_Speed'] = pd.cut(coaster_data['Top_Speed'], bins=bins,
labels=labels, right=False)
coaster_data.dropna(inplace=True)

X = coaster_data.drop("Top_Speed", axis = 1)
y = coaster_data[['Top_Speed']]
result = tree.DecisionTreeClassifier(max_leaf_nodes=5).fit(X,y)
tree.plot_tree(result, feature_names =coaster_data.drop('Top_Speed', axis =
1).columns, class_names = result.classes_, filled = True)
plt.show()
```



Drop, height, and length are the most important attributes for determining whether the roller coaster is slow (top speed of 0 to 50 mph), medium (50 to 57 mph), fast (57 to 66 mph), or the fastest (>66)

EX: a roller coaster with a height of 160 feet and a drop of 100 feet is likely to be medium. A roller coaster with a height of 80 feet and a drop of 80 feet is likely to be slow.

Practice Assignment

Algorithm Selection

Algorithm	Definition	Input / Output	Example
Linear Regression	An algorithm that can make **numeric predictions** based on another numeric attribute	• One numeric input • One numeric output	Predict the score on a math test from number of practice problems completed
Polynomial Regression	Sometimes, a polynomial fits the data better than a line.		
Logistic Regression	An algorithm that can determine the **probability** of one outcome versus another based on a numeric attribute	• One numeric input • One binary output	Predict whether someone click on a social media ad based on their age
Multivariable Logistic Regression	An algorithm that can determine the **probability** of one outcome versus another based on **multiple** numeric attributes	• Any number of numeric inputs • One binary output	Predict whether a student will pass or fail based on study hours and attendance
Decision Trees	An algorithm that can make **predictions** based on multiple attributes	• Multiple inputs (any type) • Any number of categorical outputs	Predict if a mushroom is poisonous based on its physical features

Resources

Finding Data:

- Teacher Vetted Data Sets (mostly for classwork)
- Data Sources

Structuring Projects:

- Data Question Flow Chart
- Data Question Distiller
- Attribute Planner

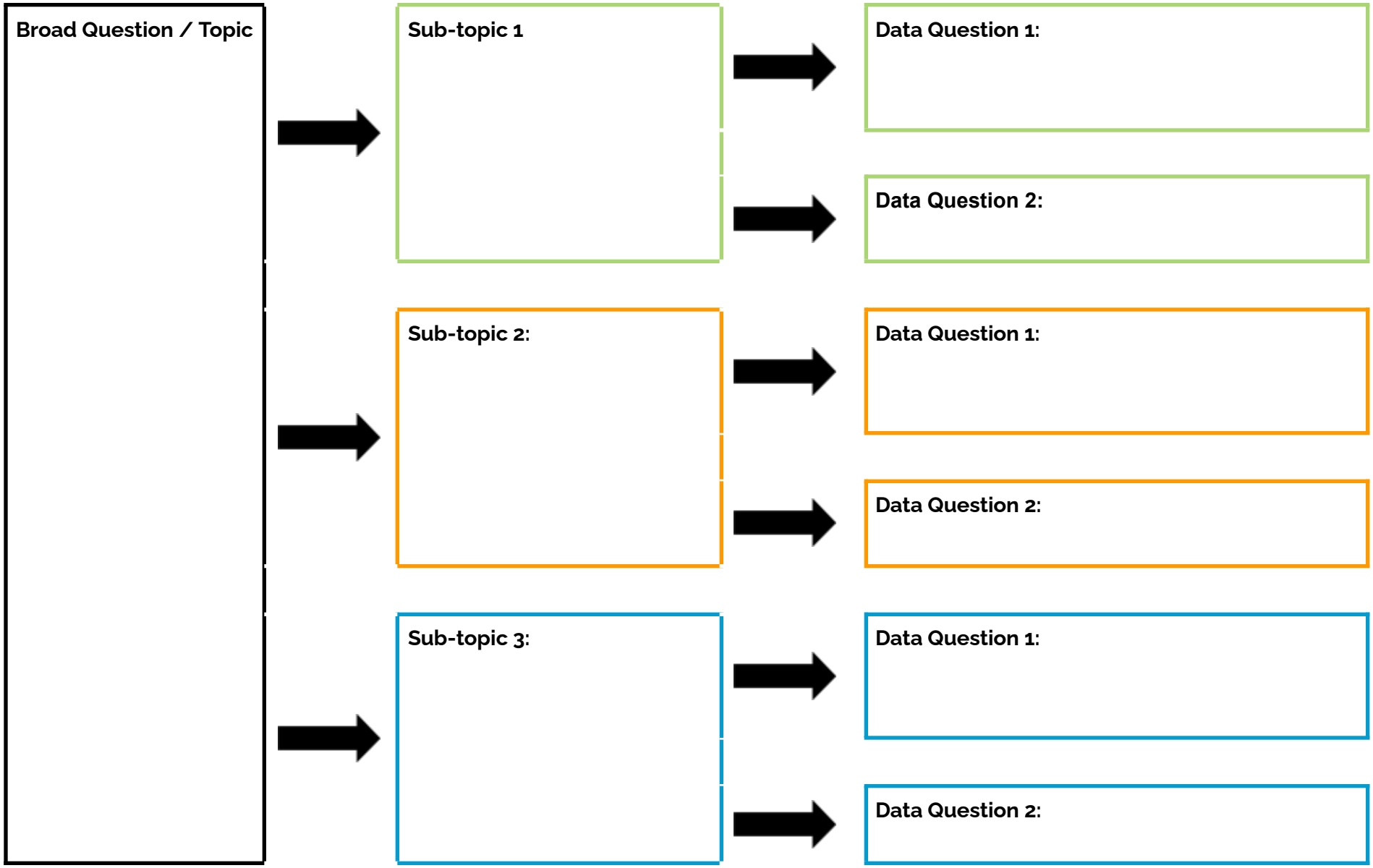
Other:

- Setting up VS code

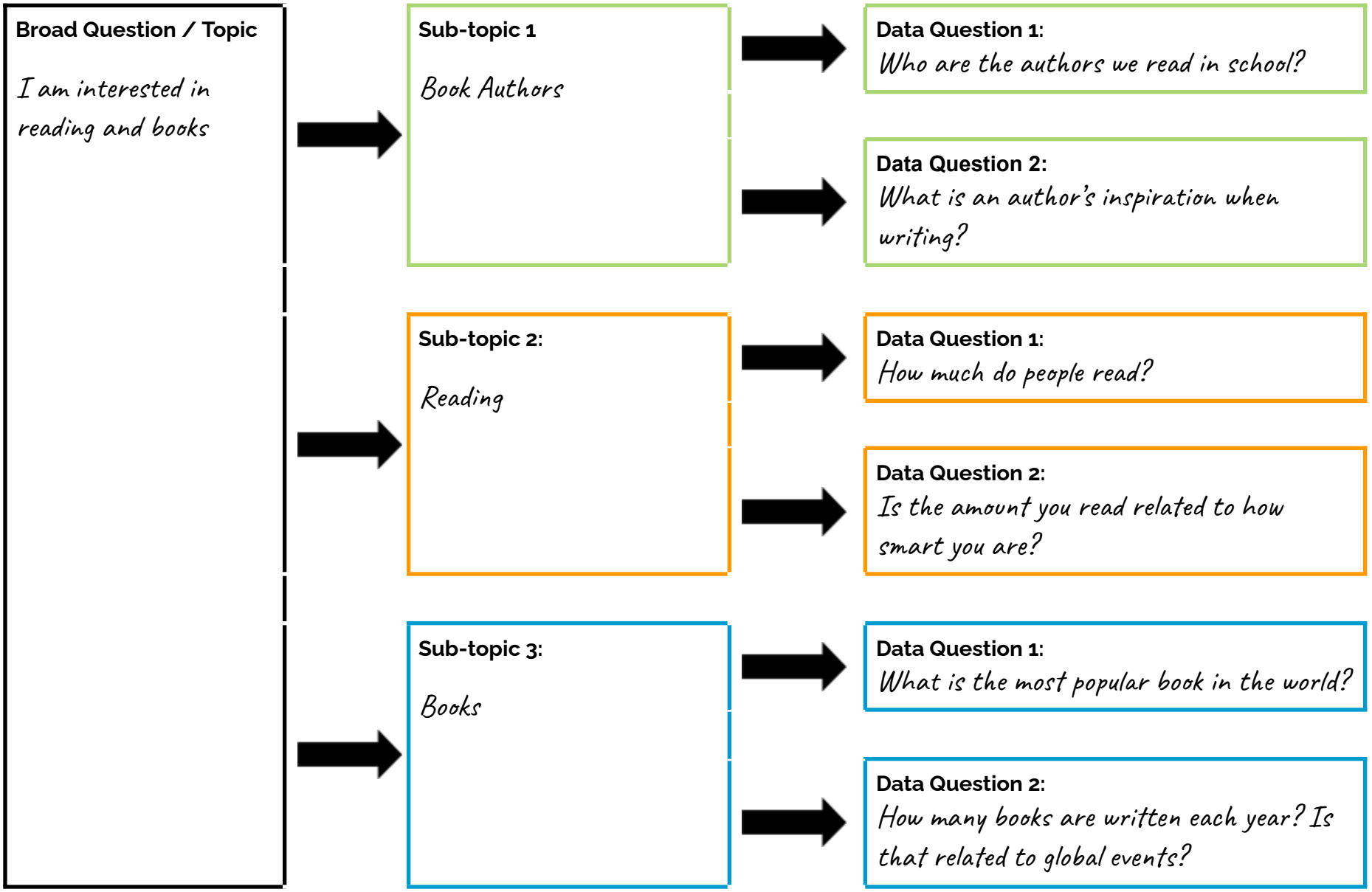
Henrico Resources:

- Clever
- Schoology
- HCPS School Calendar

Data Question Flow Chart



EXAMPLE


Broad Question / Topic
I am interested in reading and books
Sub-topic 1
Book Authors
Data Question 1:
Who are the authors we read in school?
Data Question 2:
What is an author's inspiration when writing?
Sub-topic 2:
Reading
Data Question 1:
How much do people read?
Data Question 2:
Is the amount you read related to how smart you are?
Sub-topic 3:
Books
Data Question 1:
What is the most popular book in the world?
Data Question 2:
How many books are written each year? Is that related to global events?

Data Question Distiller

Use Data Question Notes for reference

Pick your favorite data question

⬇

Is your question...

- ☐ Able to be answered with Data?
- ☐ Feasible?
- ☐ Clear?
- ☐ Objective?
- ☐ Fruitful?
- ☐ Ethical?

If you left any boxes unchecked, either rewrite your data question or choose another from your flowchart

⬇

New Data Question

⬇

Does it specify...

- ☐ Who?
- ☐ What?
- ☐ Where?
- ☐ When?

⬇

⭐Rewrite your Final Data Question⭐

Attribute Exploration

Use Types of Data Notes as reference. List each attribute, briefly describe what the attribute is and categorize the data type.

Attribute					
Description					
Type	☐ Binary ☐ Number ☐ Ordinal ☐ Categorical ☐ Qualitative	☐ Binary ☐ Number ☐ Ordinal ☐ Categorical ☐ Qualitative	☐ Binary ☐ Number ☐ Ordinal ☐ Categorical ☐ Qualitative	☐ Binary ☐ Number ☐ Ordinal ☐ Categorical ☐ Qualitative	☐ Binary ☐ Number ☐ Ordinal ☐ Categorical ☐ Qualitative

Attribute					
Description					
Type	☐ Binary ☐ Number ☐ Ordinal ☐ Categorical ☐ Qualitative	☐ Binary ☐ Number ☐ Ordinal ☐ Categorical ☐ Qualitative	☐ Binary ☐ Number ☐ Ordinal ☐ Categorical ☐ Qualitative	☐ Binary ☐ Number ☐ Ordinal ☐ Categorical ☐ Qualitative	☐ Binary ☐ Number ☐ Ordinal ☐ Categorical ☐ Qualitative

Attribute					
Description					
Type	☐ Binary ☐ Number ☐ Ordinal ☐ Categorical ☐ Qualitative	☐ Binary ☐ Number ☐ Ordinal ☐ Categorical ☐ Qualitative	☐ Binary ☐ Number ☐ Ordinal ☐ Categorical ☐ Qualitative	☐ Binary ☐ Number ☐ Ordinal ☐ Categorical ☐ Qualitative	☐ Binary ☐ Number ☐ Ordinal ☐ Categorical ☐ Qualitative

EXAMPLE

Coaster	Park	Top Speed	Max Height	Age Group	Inversions	Length	Duration
Rampage	Visionland	56	120	newest	N	3500	165
Arkansas Twister	Magic Springs and Crystal Falls	51	95	newest	N	3340	170
Big Bad John	Magic Springs and Crystal Falls	37	32	newest	N	2349	180
Giant Dipper	Santa Cruz Beach Boardwalk	55	70	Older	N	2640	112
Riddlers Revenge	Six Flags Magic Mountain	65	156	Recent	Y	4370	180

Attribute	Coaster	Park	Top Speed	Max Height
Description	Name of Roller Coaster	Park that roller coaster is in	Fastest speed of coaster (mph)	Tallest height (feet)
Type	☐ Binary ☐ Number ☐ Ordinal ☐ Categorical ✓ Qualitative	☐ Binary ☐ Number ☐ Ordinal ☐ Categorical ✓ Qualitative	☐ Binary ✓ Number ☐ Ordinal ☐ Categorical ☐ Qualitative	☐ Binary ✓ Number ☐ Ordinal ☐ Categorical ☐ Qualitative

Attribute	Age Group	Inversions	Length	Duration
Description	Old, Recent, or Newest (when it was built)	Whether there is an inversion (Y) or not (N)	Length of track (ft)	How long the ride is (sec)
Type	☐ Binary ☐ Number ☐ Ordinal ✓ Categorical ☐ Qualitative	✓ Binary ☐ Number ☐ Categorical with Order ☐ Categorical Without Order ☐ Qualitative	☐ Binary ✓ Number ☐ Categorical with Order ☐ Categorical Without Order ☐ Qualitative	☐ Predicting ✓ Number ☐ Categorical with Order ☐ Categorical Without Order ☐ Qualitative

Setting up VS Code

Part 1: Setting Up Python

1. Company Portal
    a. Download VS code from company portal
    b. Download Python 3.9.6 from company portal
2. VS code
    a. press the extensions button  on the left side. Wait a moment.
    b. Python will show up as the most popular. Press download.
3. Computer
    a. In your computer's search, type "edit environment variables for your account"
    b. Double click the one that says "`Path`" in the top section (it's usually the third in the list)
    c. Press new
    d. Type: `%APPDATA%\Python\Python39\Scripts`
    e. Press enter

Waiting for help? Steps 1 and 2 in Part 3 can be done at any point, do them while you wait.

Part 2: Installing Packages

1. In VS code, choose terminal > new terminal
2. Type: `pip install --upgrade pip seaborn matplotlib pandas statistics ipykernel scikit-learn`

If you get an error here, ask for help! There are a lot of things that could have gone wrong…

Part 3: Files

1. Create a folder on your computer (not google drive!) and call it "Data Science"
2. Download this file and save it in your data science folder
3. Create a new file: file > new file > Jupyter Notebook (.ipynb).
4. Save the file in your "Data Science" folder. Call it "Python Set Up"
5. **You must save files in VS code – they do not save automatically like in Google Drive. If you do not save them, they will disappear forever. Save frequently.**

Part 4: Wrap Up (Participation Points)

1. Raise your hand that you are finished.
2. Show your teacher your .ipynb file in VS code AND both files in your Data Science folder

If, at any point during the year, you get a new computer – Save your Data Science folder somewhere else (like Google Drive, or email it to yourself) before getting the new computer. Otherwise you will lose everything.

Data Sources for Projects

* Does not Require Login | † Requires Login to Free Account

Class Data Sets

🎓 General Public Data Portals

- Data.gov*: U.S. government open data portal across many topics.
- Kaggle Datasets †: Downloadable datasets on every topic imaginable.
- Data Commons*: Aggregated public data from major sources like NOAA and the Census.
- Statista †: Graphs and data on business, media, tech (partial free access).
- Google Dataset Search: Search engine specifically for finding datasets online.
- Pew Research*: Survey data on a variety of topics

🌎 Environment, Climate, & Weather

- NOAA Climate Data Online*: Climate records including temperature and rainfall.
- EPA Environmental Dataset Gateway*: Air/water quality and emissions data.
- Our World in Data*: Clean, global data on climate, health, and more.

⚕️ Health & Wellness

- CDC Data Tracker*: U.S. data on diseases, vaccines, and health equity.
- HealthData.gov*: Thousands of datasets on healthcare and health services.
- Kaiser Family Foundation (KFF)*: Health policy, insurance, and access stats.
- WHO Global Health Observatory*: International health statistics and trends.

📱 Technology & Digital Behavior

- Pew Research – Internet & Tech*: Survey data on digital life and tech use.
- Common Sense Media Research*: Reports and summaries about kids' media use.
- Statista – Tech: Charts on social media, device use, internet habits.

🏙️ Economy, Business, & Real Estate

- U.S. Census Data*: Demographics, education, and economic stats by region.
- Bureau of Labor Statistics*: Employment, wage, and job market trends.
- Zillow Research: Home prices, rent, and housing trends.
- Yelp Open Dataset: Business metadata and reviews, good for sentiment analysis.

🧠 Education, Youth, & Behavior

- CDC Youth Risk Behavior Survey*: Teen behavior data—screen time, sleep, grades.
- National Center for Education Statistics*: U.S. education data by age and region.
- Harvard Dataverse: Open-access social science datasets.

🏈 Sports & Performance

- MaxPreps: High school sports stats and scores.
- Sports Reference*: College and pro data for most major sports.
- Kaggle Sports Datasets: Performance and team-based data.

🧪 Survey or Student-Collected Data

- Google Forms*: Make and analyze student surveys.
- SurveyMonkey: Create simple surveys online.
- Microsoft Forms*: Microsoft version of Google Forms.

🏛️ Government & Politics

- FEC Data*: Campaign contributions and finance records.
- GovTrack*: Congressional votes and bill tracking.
- OpenSecrets: Money in politics: lobbying and donations.
- Ballotpedia*: Election results and candidate info.
- Census Voting & Registration*: Voter turnout and registration demographics.
- Pew Research – Politics*: Surveys on political opinions and trends.
- FiveThirtyEight Datasets: Politics, polling, and media datasets.

🎬 Arts, Entertainment, & Pop Culture

- IMDb Datasets †: Movie metadata, cast, and ratings.
- Kaggle Music/Spotify: Streaming trends and song characteristics.
- TMDb API †: Movie/show data via API (requires key).
- YouTube Trending Videos: View counts and tags of viral videos.

🌍 Social Justice & Equity

- Mapping Police Violence*: Police use of force by race and location.
- Bureau of Justice Statistics*: Crime and incarceration by demographics.
- UN Data*: Global equity and human development indicators.
- World Bank Gini Index*: Measures of income inequality by country.
- Human Rights Campaign: LGBTQ+ reports and advocacy data.

🐶 Animals & the Natural World

- iNaturalist*: Wildlife sightings and biodiversity data.
- USDA APHIS: Animal health and agriculture data.
- NOAA Fisheries*: Marine species and conservation data.
- Petfinder API*: Pet adoption trends (API).
- Smithsonian National Zoo*: Animal profiles and care data.

🎮 Gaming & eSports

- Kaggle Video Game Sales: Console and game title sales data.
- TwitchTracker †: Livestream platform analytics.
- SteamCharts †: Game popularity over time on PC.
- League of Legends API †: Advanced game data access.
- Newzoo (summaries) †: Reports on global gaming markets.

🛍️ Fashion, Beauty, & Consumer Behavior

- Google Trends*: Compare search terms over time.
- Pinterest Trends*: What's trending in fashion/beauty.
- BEA Spending Data: Personal spending categories over time.
- Statista – Fashion & Beauty: Consumer behavior and product stats.

🏠 Housing, Urban Development, & Community

- HUD User Data*: Urban housing and development programs.
- Zillow Research: Home value and rent trends.
- Local Open Data Portals*: City-level housing and planning info.
- American Housing Survey*: Census data on home conditions.
- Redfin Data Center*: Market competition and migration trends.